Carnegie Mellon University

CARNEGIE INSTITUTE OF TECHNOLOGY

THESIS

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE DEGREE OF Master of Science

TITLE _____

_____

PRESENTED BY _____

ACCEPTED BY THE DEPARTMENT OF

Information Networking Institute

_____ THESIS ADVISOR _____ DATE

_____ ACADEMIC ADVISOR _____ DATE

_____ DEPARTMENT HEAD _____ DATE

APPROVED BY THE COLLEGE COUNCIL

_____ DEAN _____ DATE

Tool-Orchestrated Agentic Reverse Engineering and Malware Analysis

Submitted in partial fulfillment of the requirements for

the degree of

Master of Science

in

Information Security

Matthew Ung

B.S. in Computer Science, Tufts University, 2024

Carnegie Mellon University
Pittsburgh, PA

April, 2026

© Matthew Ung, 2026
All Rights Reserved

Acknowledgements

I am grateful to the faculty, peers, collaborators, friends, and family members who supported this work. I owe particular thanks to my advisors Jeffrey Gennari and Hanan Hibshi for their sustained feedback throughout the process. Their guidance, technical discussion, and patience made it possible to carry the project from an exploratory tooling effort into a thesis with a clearer research question, a reproducible evaluation path, and a more defensible set of claims.

Abstract

Malware analysis through reverse engineering remains a labor-intensive process that requires analysts to combine evidence from disassembly, decompilation, string and configuration recovery, capability triage, and related artifact inspection. In practice, analysts move among several tools and intermediate representations rather than relying on a single unified view of the binary, which makes the problem difficult to automate cleanly [1, 2, 3]. Although large language models have improved substantially in code understanding and tool use, it remains unclear how well they can support binary analysis when the relevant evidence is fragmented across specialized tools rather than contained in source code. Public reporting and federal complaint data further suggest that AI-enabled cyber operations are already moving beyond laboratory demonstrations [4, 5].

This challenge is especially apparent in malware analysis, where samples routinely employ evasion and obfuscation techniques designed to hinder analysis, including anti-analysis checks, packed or encrypted artifacts, staged behavior, dynamically generated code, and environment-sensitive execution [1, 6]. Thus, malware reverse engineering serves as a demanding stress test for tool-using LLM systems, because RE success routinely depends on structured evidence gathering, cross-tool synthesis, and flexible reasoning capabilities for handling uncertainty.

This thesis explores whether tool-orchestrated agentic workflows can improve reverse-engineering performance relative to weaker or less structured baselines. The implemented system uses a staged pipeline with a light preparatory stage, deterministic presweeps, planning, specialized workers, optional validator review, and final reporting rather than a single monolithic agent. These stages have structured access

to Ghidra, static-analysis utilities, unpacking support, and artifact-recovery services, following an analysis-first design closer to program-analysis-assisted approaches such as ReSym than to text-only prompting [7, 8, 9]. The goal is to study how staged reasoning, explicit tool access, and evidence review affect tasks such as behavior reconstruction, configuration decoding, packing analysis, and malware-oriented triage.

To make the workflow both usable and measurable, the thesis studies the same pipeline in two modes: an analyst-facing interface for interactive inspection of pipeline progress and tool activity, and an evaluation harness that runs controlled sample-task experiments with structured judging and graph generation. In the final held-out comparison against five blind malware-simulation executables, no tested alternative achieved a statistically significant overall improvement over the maintained baseline, but executable-level results were heterogeneous enough that average-only reporting would have hidden meaningful sample-specific differences. The final evidence therefore supports a narrower conclusion: in this setting, agentic reverse engineering behaves more like a workflow-selection and robustness problem than a search for one universally best configuration.

Table of Contents

Acknowledgements **iv**

Abstract **v**

1 Introduction **1**

1.1 Problem Motivation 2

1.2 Research Questions 2

1.3 Thesis Scope and Expected Contributions 2

1.4 Thesis Organization 3

2 Background and Problem Setting **4**
2.1 Reverse Engineering as an Evidence-Gathering Problem 4
2.2 LLM Agents, Tool Use, and Orchestration 5
2.3 MCP-Style Tool Integration 6
2.4 Binary Analysis Tooling in Scope 7
2.5 Problem Formulation for the Thesis 7

3 Related Work **8**
3.1 AI-Assisted Reverse Engineering and Malware Analysis 8
3.2 Tool-Using Agents and Multi-Agent Orchestration 10
3.3 Program Analysis Toolchains and Human Analyst Workflows 11
3.4 Evaluation of Security and Reverse-Engineering Systems 12
3.5 Thesis Positioning 12

4 System Overview **13**
4.1 End-to-End Workflow 13
4.2 Planner, Worker, Validator, and Reporter Roles 14
4.3 Configuration Surfaces 14
4.4 Shared State and Evidence Flow 15
4.5 Why This Architecture Fits Reverse Engineering 16

5 Implementation and Tool Integration **17**
5.1 System Organization and Execution Path 17
5.2 Workflow Configuration 18
5.3 Ghidra Integration 18
5.4 Supporting Static Tooling and Artifact Services 19
5.5 Artifacts, Bundles, and Provenance 20
5.6 User-Facing Monitoring and Result Inspection 20

6 Evaluation Methodology and Experimental Design **22**
6.1 Evaluation Goals 22
6.2 Benchmark Corpus and Task Model 22

6.3 Single Runs, Sweeps, and Repetitions 23
6.4 Broad Coverage and Final Held-Out Malware Evaluation 24
6.5 Baselines and Experimental Variables 25
6.6 Judging and Scoring 26
6.7 Statistical Treatment and Comparison Strategy 26

7 Results and Analysis **27**
7.1 Development-Stage Archive Evidence 27
7.1.1 Broad Coverage Pilot 28
7.1.2 Focused Decoder-Depth Case Study 29
7.2 Final Held-Out Evaluation 30
7.2.1 Overall Performance After Recovery 31
7.2.2 Executable-Level Heterogeneity 33
7.2.3 Efficiency and Repeated Tool Use 34
7.2.4 Robustness Caveat from the Original Held-Out Sweep 37
7.3 Discussion and Claim Boundaries 37
7.3.1 Findings Directly Supported by the Data 37
7.3.2 Plausible Interpretations 38
7.3.3 Open Uncertainties and Measurement Gaps 38

8 Limitations and Threats to Validity **41**
8.1 Benchmark Scope and External Validity 41
8.2 Statistical Power and Repetition Count 42
8.3 Model Scope 42
8.4 Recovery and One-Shot Robustness 43
8.5 Tool-Call Redundancy Measurement 43
8.6 Judging and Measurement Proxies 45
8.7 Operational and Ethical Considerations 45

9 Conclusion and Future Work **46**
9.1 Conclusion 46
9.2 Future Work 48

Bibliography **50**

Appendix A Supplementary Evaluation Material **53**

A.1 Sample and Variant Traceability 53

A.1.1 Broad Coverage Pilot Executables 54

A.1.2 Focused Decoder-Depth Executables 55

A.1.3 Final Held-Out Executables 56

A.1.4 Maintained Variant and Configuration Labels 56

A.2 Judge and Prompt Artifacts 57

A.3 Configuration and Experiment Provenance 58

A.4 Held-Out Failure Accounting 60

A.5 Detailed Final Per-Executable Panels 60

A.6 Reproduction Commands 62

1

Introduction

Software reverse engineering often requires analysts to combine decompilation, string recovery, capability triage, configuration extraction, and other artifact-driven inspection steps. That workflow is tool-rich and evidence-driven, but it is also time-consuming and difficult to automate well. This thesis studies whether an agentic system that explicitly orchestrates multiple tools can produce better binary understanding than weaker baselines that rely on simpler prompting or less structured analysis flows. The question is increasingly relevant because public reporting and federal complaint data suggest that AI-enabled cyber operations and AI-assisted cybercrime are already appearing outside laboratory demonstrations [4, 5].

The system presented in this thesis is centered on Model Context Protocol (MCP)-style tool orchestration rather than a single monolithic agent. In its maintained form, the workflow combines a light preparatory stage, deterministic presweeps, planning, specialist workers, optional validation, and final reporting, with structured access to Ghidra, supporting static-analysis utilities, and artifact-recovery services. The thesis therefore addresses both systems questions and empirical questions: how such a workflow should be designed for reverse engineering, why those design choices matter,

and whether they measurably improve task performance.

1.1 Problem Motivation

Reverse-engineering tasks are not homogeneous. Some center on control-flow and structural reasoning, while others depend more heavily on recovered strings, decoded configuration data, or packer handling. Because useful evidence is distributed across many tools and intermediate artifacts, and no single model invocation can gather and cross-reference all of it reliably, single-shot prompting is poorly matched to this setting. Tool-augmented agents may improve coverage and evidence quality, but they also introduce orchestration complexity, new failure modes, increased cost, and reproducibility concerns that should be studied rather than assumed.

1.2 Research Questions

1. Does a tool-orchestrated agentic workflow improve binary-analysis task performance relative to weaker baselines?

2. Which workflow surfaces appear to matter most in the evaluated experiments: worker specialization, worker prompt shape, response scope, or architecture emphasis?

3. How consistent are those effects across executables and task types, and when do overall averages hide heterogeneous behavior?

4. What robustness and orchestration failure modes remain even when the workflow is tool-augmented?

1.3 Thesis Scope and Expected Contributions

This thesis makes four contributions:

1. A modular agentic reverse-engineering system that separates preparation, planning, specialist evidence collection, optional review, and reporting so those roles can be varied and measured explicitly.

2. MCP-mediated integration of Ghidra, supporting static-analysis tools, and artifact-recovery services, providing structured tool access rather than ad hoc manual interaction.

3. A binary-analysis evaluation harness built around sample-task pairs, baseline-first sweeps, repeated runs, and judge-driven scoring, designed to make workflow comparisons reproducible.

4. A focused empirical investigation of whether tool-augmented agents outperform weaker baselines on reverse-engineering and malware-understanding tasks, with attention to both aggregate performance and executable-level heterogeneity.

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 introduces the reverse-engineering problem setting and the technical background needed for the thesis. Chapter 3 positions the work against prior literature on AI-assisted reverse engineering, malware analysis automation, and tool-using agents. Chapter 4 describes the system at a high level, and Chapter 5 details the implementation and tool integrations. Chapter 6 defines the experimental methodology. Chapter 7 presents quantitative and qualitative findings. Chapter 8 discusses limitations and threats to validity, and Chapter 9 concludes with future work.

2

Background and Problem Setting

2.1 Reverse Engineering as an Evidence-Gathering Problem

As briefly discussed earlier, reverse engineering is not a single question answered by a single tool. Analysts move among disassembly, decompilation, string inspection, structural triage, pattern matching, and artifact correlation in order to construct an evidence-backed explanation of program behavior [2, 3]. This makes the domain a strong candidate for studying tool-orchestrated agents.

In practice, reverse-engineering tasks often require both broad survey steps and narrow recovery steps within the same analysis session. Malware-oriented questions also demand conservative claim boundaries, explicit provenance tracking, and visible uncertainty handling. Importantly, the quality of an answer depends on both the final narrative and whether the system gathered the right supporting artifacts along the way.

In the evaluation benchmark, those questions are operationalized as recurring task categories rather than a single generic "analyze the binary" prompt. The broadest task family, *default analysis*, asks for program-level behavior reconstruction with

evidence anchors and uncertainty handling. Focused tasks then narrow the scope to a single mechanism or artifact family, including configuration or value recovery, decode or parser-flow reconstruction, packing or unpacking triage, and anti-analysis or deception audit. This distinction matters because the evaluation checks not only whether the workflow can produce a plausible narrative but also whether it can pivot from broad overview to targeted recovery when the task demands deeper evidence.

The thesis also uses a concrete notion of acceptable evidence. A strong binary-analysis claim should be tied to identifiable artifacts such as function names or addresses, API or import references, decoded strings or configuration values, registry paths, service names, command lines, or tool outputs such as FLOSS recoveries, capa rules, packer detections, or unpacked-binary metadata. Control-flow explanations are treated as stronger when they connect those artifacts to the branch structure or data path that actually enables the behavior. Reports are penalized when they describe a capability class without showing where the conclusion came from, or when they fail to distinguish observed behavior from inference. That standard later appears directly in the structured judging rubric through dimensions such as `evidence_grounding`, `specificity`, and `false_claim_control`.

2.2 LLM Agents, Tool Use, and Orchestration

The system studied here is explicitly modular. Rather than expecting one model invocation to reason, search, inspect artifacts, and self-critique all at once, the workflow decomposes the task into stages with different responsibilities. This decomposition fits reverse engineering because the task naturally alternates between planning, evidence collection, validation, and synthesis. It also aligns with broader agent work showing that long-horizon performance can benefit from separating high-level planning from low-level execution, that agent-computer interface design can materially affect what the model is able to do in practice, and that iterative critique can improve

outputs when review is made explicit [10, 11, 12, 13].

In this design, preparatory stages gather reusable context, planner stages decompose the query and allocate work across available specialists, and worker stages gather concrete evidence through bounded tool use. Validator stages, when enabled, review coverage, evidence quality, and missing artifacts before the result is finalized. Reporter stages synthesize the resulting findings into the final narrative artifact used by the harness and judge.

2.3 MCP-Style Tool Integration

The system uses MCP-style servers to expose tool capabilities to the agent runtime. This design choice matters because the thesis argument is partly about interface design: how tool affordances are surfaced to agents, how outputs are normalized, and how evidence is routed back into shared state.

As evaluated in this thesis, the operational tool surface is organized around a static-analysis lane plus an artifact-management layer. The static lane provides access to Ghidra and supporting utilities for strings, decoded-string recovery, capability triage, hash resolution, YARA, and binwalk-style file inspection. The artifact layer manages prepared bundles, manifests, run records, judge outputs, and experiment aggregates. The runtime code also reserves metadata for dynamic-tool partitioning, but the active server registry does not currently provide VM, debugger, process-monitor, packet-capture, or other execution-oriented MCP servers.

For the experiments in this thesis, the system is therefore static and artifact-backed. Execution-oriented analysis is not part of the implemented evaluation path, and the stage prompts are written to treat active sample execution as out of scope unless such support exists explicitly.

2.4 Binary Analysis Tooling in Scope

The technical role of each major tool class matters because later architecture chapters build directly on these capabilities.

Ghidra provides decompilation, disassembly, symbol context, and structural inspection, and serves as the primary backend for control-flow and data-flow reasoning. String and FLOSS-style tooling surface literal and decoded text evidence that can anchor behavioral claims. Capability and signature tools provide fast high-level hints that can be corroborated by deeper analysis in later stages. The current implementation does not include an operational dynamic-analysis MCP toolchain, so the evaluated workflow stops at static and artifact-backed evidence rather than live execution or runtime tracing.

2.5 Problem Formulation for the Thesis

At a high level, each experiment asks whether a workflow configuration can answer a binary-analysis task better than a baseline. The atomic evaluation unit is a sample-task pair. That framing carries forward into the evaluation chapter as the basic comparison unit for both authored and held-out experiments.

Each evaluation unit takes as input a binary sample, a task prompt, the available tool set, a workflow configuration, and a model configuration. It produces as output a final analysis report, intermediate evidence artifacts, a judge score, and run metadata. Comparisons are then made along several axes: score deltas, task success rates, error rates, and qualitative failure analysis across matched sample-task pairs.

3

Related Work

This chapter positions the thesis against three overlapping research areas: AI-assisted reverse engineering and malware analysis, autonomous or semi-autonomous software agents, and binary-analysis evaluation methodology. The point is not simply to collect related citations, but to clarify what technical gap this thesis addresses.

3.1 AI-Assisted Reverse Engineering and Malware Analysis

Recent academic work addresses several pieces of the design space explored in this thesis. The malware-analysis SoK by Kunwar et al. [1] shows that transformer-style methods are now applied across static, dynamic, and hybrid malware tasks while emphasizing unresolved problems around obfuscation, explainability, and evaluation. ReSym [9] is especially relevant because it demonstrates an analysis-first pattern in which deterministic binary facts are gathered first and then handed to an LLM for higher-level symbol and structure recovery. JTrans [14], while focused on binary similarity rather than malware triage, reinforces the broader lesson that control-flow structure matters and that plain token streams are often too weak a representation for difficult binary tasks. Together, these papers justify treating binary analysis as

a structured evidence-integration problem rather than a pure prompt-engineering problem.

Benchmark-centric work is also becoming more visible in this area. BinMetric introduces a broad binary-analysis benchmark with six task families and uses it to compare model performance on decompilation-adjacent tasks, summarization, and assembly-oriented outputs [15]. That work provides useful context, but it serves a different purpose: BinMetric is primarily a model-capability benchmark, whereas the present thesis studies a full workflow with staged tool use, shared evidence flow, and configuration-level comparisons over complete sample-task analyses. Very recent contemporaneous work by Hu et al. studies LLM-based pseudocode deobfuscation with a dedicated deobfuscation benchmark and focuses on how reasoning strength, fine-tuning, and obfuscation severity affect deobfuscation quality [16]. That paper provides relevant positioning for obfuscation-heavy reverse engineering, but it is better read as adjacent evidence about deobfuscation-specific model behavior than as a direct precursor to the workflow design studied here.

Recent practitioner-facing systems already point toward the same direction. GhidrAssist [17], for example, embeds language-model assistance directly into Ghidra, while GhidraMCP [7] exposes a structured tool boundary between Ghidra and external agents. These systems are useful comparators because they demonstrate growing interest in LLM-assisted reverse engineering, but they do not by themselves answer the workflow question studied in this thesis: whether planning, specialist decomposition, validation, and experiment-grade measurement materially change performance and robustness within a staged analysis workflow. Raitsis et al. [6] add a complementary perspective by surveying obfuscation types and detection challenges, which reinforces why malware-facing reverse engineering is a more demanding target than clean binary similarity or source-adjacent code tasks.

3.2 Tool-Using Agents and Multi-Agent Orchestration

The orchestration structure in this thesis is motivated more by the operational needs of reverse engineering than by any claim that one general-purpose agent architecture is already settled in the literature. Reverse engineering naturally decomposes into planning, evidence collection, review, and synthesis, and recent agent systems work suggests that interface design, planning structure, and topology can materially change outcomes. SWE-agent argues that language-model agents benefit from task-specific agent-computer interfaces and shows that interface design can materially affect performance on software engineering tasks [11]. Plan-and-Act reports gains from separating high-level planning from low-level execution on long-horizon tasks [10]. Multi-Agent Design studies prompt and topology choices jointly and concludes that both contribute materially to multi-agent effectiveness [18]. Abuzakuk et al. further treat workflow optimization as a systems problem, using execution traces to compile recurring tool sequences into deterministic meta-tools that reduce LLM calls and improve robustness on their benchmarks [19]. These works do not study reverse engineering directly, but they support evaluating workflow topology and orchestration structure as real design choices rather than presentation details.

The validator and review aspects of this thesis also overlap with broader critique-and-revision work. Self-Refine studies iterative feedback and refinement using the same model as generator, critic, and refiner [12]. CRITIC extends that idea by incorporating external tools into the critique loop so that revisions can be guided by tool-derived feedback rather than only internal self-evaluation [13]. These papers are not reverse-engineering papers, and the thesis implementation uses explicit validator stages rather than generic self-refinement prompting, but they do support treating critique and revision loops as architectural design choices rather than as incidental prompt embellishments.

Prompt-surface changes likewise should not be assumed to help automatically. Basil et al.'s Prompting Science Report 4 finds that expert-persona prompts did not consistently improve factual accuracy on difficult objective QA benchmarks and sometimes degraded it when they were mismatched to the task [20]. That result is not a reverse-engineering result, but it reinforces the thesis decision to treat role-prompt and prompt-shape changes as experimental variables rather than default upgrades.

3.3 Program Analysis Toolchains and Human Analyst Workflows

The human-analyst workflow and interface literature is equally important. Mattei et al. [2] show that many reverse-engineering tools fail to present results in code context or to support the phases analysts actually move through, while Yakdan et al. [3] show that readability-oriented transformations can materially improve analyst performance on malware tasks. At a more structural level, Schwartz et al. [21] demonstrate the value of explainable program reasoning through rule-based recovery of C++ classes and methods. These works collectively support several design choices in this thesis: keep evidence tied to code context, favor readability and reversible interpretation aids, and preserve deterministic traces or validation logic wherever possible.

The surrounding analyst ecosystem also matters. In addition to decompiler-centric environments, real workflows commonly rely on file- and packer-identification utilities, hash-resolution services, and malware-intelligence feeds. The thesis implementation uses tools in that general mold, including services such as HashDB [22], file-type utilities such as Detect It Easy [23], and evaluator-side malware-intelligence references such as MalwareBazaar [24]. These tools do not replace structural reverse engineering, but they help shape how analysts prioritize effort, validate hunches, and connect local binary evidence to broader context.

3.4 Evaluation of Security and Reverse-Engineering Systems

The evaluation contribution in this thesis is also narrower than a full benchmark survey. Recent work such as BinMetric [15] is useful because it expands benchmark coverage for binary-analysis question answering and model comparison. The evaluation harness in this thesis serves a different goal: it focuses on sample-task pairs, baseline-first sweeps, repaired experiment provenance, and rubric-driven judging designed for evidence-backed binary-analysis reports. The goal is not to claim a universal evaluation standard for agentic reverse engineering, but to make the tested workflow reproducible enough that prompt-shape, topology, and architecture changes can be compared without losing the underlying task context or artifact trail.

3.5 Thesis Positioning

The distinctive claim of this thesis is not merely that language models can help with reverse engineering. The system contribution is that explicit tool orchestration, staged specialization, and evidence review can be treated as first-class design elements in a reverse-engineering workflow, and that those choices can be evaluated systematically rather than anecdotally.

4

System Overview

This chapter gives a high-level view of the system before the implementation details appear. The emphasis is on workflow, component roles, and evidence flow rather than source-tree organization.

4.1 End-to-End Workflow

A run begins with a user query or evaluation task, resolves a workflow configuration, provisions tool access, and then executes a staged analysis pipeline. The important point is not the exact call chain but the way configuration, tooling, and staged reasoning are assembled into one repeatable workflow. Appendix A.6 provides the concrete launch details for readers who need the artifact-level execution path.

In the maintained baseline, execution begins with a light preparatory pass, called *preflight* in the system implementation, and deterministic presweeps that gather reusable context before any broad free-form reasoning occurs. A planner then converts the request and shared context into bounded work items, specialist workers execute those items with tool access appropriate to their role, and a reporter synthesizes the result into the final analysis artifact. Some experimental pipelines insert an

explicit validator stage between workers and reporting, but validator review is not universal across all runs. After the final report is produced, the harness captures artifacts, invokes judging, and aggregates results into the experiment record.

4.2 Planner, Worker, Validator, and Reporter Roles

The workflow supports multiple stage kinds, but its core pattern is straightforward: prepare context, decompose the task, gather evidence, optionally review coverage, and then synthesize the final answer.

Preparatory stages establish basic sample metadata and reusable static-analysis outputs. Planners decide how to decompose the task and which evidence should be gathered. Workers perform focused analysis passes, often with specialized tool access or role prompts that constrain their scope. Validators, when present, act as an internal quality gate by reviewing coverage, evidence quality, and missing artifacts before final reporting. Reporters convert intermediate evidence into the final narrative artifact used by the harness and judge.

4.3 Configuration Surfaces

The system exposes several configuration families that are central to the evaluation:

- **Response-scope variants** control how concise or expansive the final report should be.
- **Worker subagent profiles** determine whether workers are generalists or specialists.
- **Worker prompt shape and role-prompt mode** tune the instruction surface given to each worker.
- **Tool profiles** control which MCP tools are available to the pipeline.

- **Architecture presets** define the worker mix.
- **Pipeline presets** select the stage sequence.
- **Validator topology and review level** determine how strict the internal quality gate is.

These are not merely engineering options; they are the intervention points used by the evaluation harness to compare baseline and variant behavior. Some of them primarily change how work is organized, such as worker subagent profiles, architecture presets, and pipeline presets. Others change how evidence is requested or summarized, such as response-scope variants and worker-prompt surfaces. Tool profiles matter most when asking how much performance depends on breadth of tool access rather than on orchestration alone. The full mapping of variant labels to configuration changes is provided in Table Table 1.4.

4.4 Shared State and Evidence Flow

The system maintains shared run state so that later stages can inspect earlier findings, tool outputs, and failure metadata. This matters because reverse engineering benefits from explicit provenance: shared state determines what later stages can trust, what evidence survives handoff between agents, and what ultimately appears in final reports and judge inputs.

Shared state captures four categories of information: intermediate tool outputs and structured findings from earlier stages; stage status logs and work-item tracking; the final analysis report and supporting artifacts; and run metadata used by the evaluation harness, including bundle provenance and comparison context.

4.5 Why This Architecture Fits Reverse Engineering

The same shared-state model also supports the two user-facing surfaces discussed later in the thesis. The analyst-facing interface exposes pipeline progress, tool logs, and intermediate stage outcomes while a run is in progress. The evaluation-facing browser exposes archived runs, repaired comparisons, and per-executable result views after the fact. Those interfaces matter scientifically because they make provenance inspectable rather than leaving the workflow as a single opaque report.

The value proposition of the architecture is straightforward: binary analysis benefits from staged specialization because the evidence sources are heterogeneous, and high-confidence answers require both discovery and disciplined synthesis. A monolithic prompt can mention these needs, but it cannot enforce them structurally. The architecture used here makes those roles explicit so they can be inspected, varied, and measured.

5

Implementation and Tool Integration

This chapter translates the high-level workflow into concrete implementation choices. The focus is on how the architecture is realized in software, how tools are exposed to the runtime, and how artifacts and experiment records are preserved. Detailed file-by-file traceability is collected in the appendix so that the main discussion can stay at the system level.

5.1 System Organization and Execution Path

The implementation separates the research system from the measurement apparatus used to study it. The core workflow handles configuration, tool loading, worker topology assembly, staged pipeline execution, and shared-state management. The evaluation infrastructure, described in Chapter 6, handles bundles, judging, sweeps, results aggregation, and recovery. This separation matters because it keeps workflow logic independent of the harness mechanics, making it possible to vary one without disturbing the other. Appendix A.6 and the supplementary provenance material map these ideas back to the concrete artifact layout for readers who need code-level traceability.

5.2 Workflow Configuration

The workflow uses declarative configuration for stage contracts, worker archetypes, architectures, prompts, and pipeline selection. This design is deliberate: it allows ablations and reproducible experiments to be defined by controlled configuration changes rather than by modifying runtime logic.

Pipeline presets determine which stages run and in what order. Architecture presets determine the available worker mix. Worker prompt shape, role-prompt mode, and review level tune behavior without modifying the core runtime, keeping ablation definitions declarative and auditable.

5.3 Ghidra Integration

Ghidra is the primary structural-analysis backend in the system. What matters here is not only that it is used, but how it is used: through prepared bundles, exported analysis artifacts, and agent-facing access mediated by explicit tooling rather than ad hoc manual interaction.

Ghidra-backed analysis is prepared through bundle generation, which precomputes decompilation and structural metadata before the agent pipeline begins. Agent stages then consume these Ghidra-derived artifacts, and where allowed, they can also query Ghidra-facing tools directly during execution. The system explicitly tracks whether analysis is occurring on the original sample or on a derived artifact such as an unpacked UPX binary so that provenance is preserved across unpacking and re-analysis steps.

At the integration boundary, this work is motivated less by generic chatbot assistance and more by prior evidence that reverse-engineering support tools should improve readability, preserve analyst context, and retain explainable structure [3, 21]. Concretely, the system is closest to Ghidra's plugin-mediated extension model [8]

and to public bridges such as GhidraMCP [7]. It is also adjacent to assistant-style plugins such as GhidrAssist [17], although the emphasis here is not only in-editor assistance. The thesis system uses Ghidra as one component inside a broader staged workflow and evaluation harness, with explicit result capture, replayable bundles, and controlled tool mediation.

5.4 Supporting Static Tooling and Artifact Services

The broader tool ecosystem matters because much of the thesis is about tool augmentation rather than raw model capability.

Static support tools include strings, FLOSS, and hash lookups, while additional utilities provide capability extraction, YARA, binwalk, and UPX-oriented support. Tool profiles allow the evaluation to measure how much accuracy depends on the breadth of this static tool ecosystem by comparing full-access runs against restricted-tool configurations. The runtime architecture reserves metadata for distinguishing static from dynamic tool domains, but the active system registers only static-analysis, enrichment, patching, and artifact-management servers. As a result, the evaluated system does not perform live detonation, debugger orchestration, packet capture, process monitoring, or other execution-driven analysis.

Some of the surrounding tool ecosystem is directly integrated, while other pieces serve mainly as reference points for the analyst workflow the system is trying to emulate. This choice is not only practical; it also follows reverse-engineering usability work showing that analysts benefit from multi-phase workflows, visible evidence transitions, and readable context rather than single opaque outputs [2, 3]. HashDB [22] is representative of hash-resolution services that help connect encoded imports or constants to likely APIs. Detect It Easy [23] is representative of file-typing and packer-identification utilities that help scope unpacking or triage effort. MalwareBazaar [24] is representative of external malware-intelligence feeds that can con-

textualize hashes and sample provenance, even though the thesis experiments keep evaluator-side corpus control separate from any live feed dependency.

5.5 Artifacts, Bundles, and Provenance

Tool integration is therefore only one part of the implementation story. The other part is visibility. The runtime records which tools were exposed, which ones were called, how outputs were routed into shared state, and which derived artifacts were retained for later stages or for evaluation. That record is what makes repaired experiments, tool-usage summaries, and failure analysis possible. Without it, the workflow would still produce reports, but it would be much harder to determine whether a weak result reflected poor reasoning, poor tool selection, missing evidence, or an orchestration failure.

Prepared bundles and run artifacts are central to the system's reproducibility story. Bundles capture analysis inputs and precomputed artifacts for a sample, while run records capture the outputs of one particular workflow configuration.

Bundles are reused when their generation inputs remain fresh, and prompt or persona changes do not by themselves invalidate a bundle. Run records preserve final reports, judge outputs, live status, and per-task metadata for each execution. Recovery and repaired-experiment workflows preserve provenance instead of mutating original results in place, ensuring that the distinction between first-pass and recovered runs remains visible in the experiment record.

5.6 User-Facing Monitoring and Result Inspection

The system also includes monitoring and browsing interfaces that operationalize the provenance model and make failure analysis tractable. These interfaces expose in-progress status, archived experiment summaries, per-executable views, and recovery outputs. Their scientific role is not merely convenience: they make it possible to

inspect incomplete runs, distinguish analytical weakness from orchestration failure, and preserve the provenance needed for repaired comparisons.

6

Evaluation Methodology and Experimental Design

This chapter explains how the thesis evaluates the system. The evaluation methodology is grounded in the implemented binary-analysis harness and is designed to keep claims aligned with the available evidence.

6.1 Evaluation Goals

The evaluation has three goals. The first is to measure whether tool-orchestrated workflows outperform weaker baselines on binary-analysis tasks. The second is to compare the workflow surfaces that remain central in the evaluated experiments: topology, prompt shaping, response scope, and architecture emphasis. The third is to capture both successful analyses and structured failure modes such as analysis errors, validator blocks, judge failures, and repeated-tool-use hotspots, so that the results reflect operational reliability and efficiency alongside analytical quality.

6.2 Benchmark Corpus and Task Model

The binary benchmark uses an experimental corpus of authored executables and executable variants. The evaluation unit is a sample-task pair rather than only a

sample. This detail is important because it lets the thesis distinguish broad understanding tasks from focused recovery tasks. That design differs from benchmark-centric evaluations such as BinMetric [15], which are primarily organized around fixed binary-analysis question sets for model comparison. Here the target of evaluation is the configured analysis workflow acting on a sample-task unit, with intermediate artifacts and operational failures preserved alongside the final score.

Sample variants cover conditions such as stripping, packing, deceptive strings, anti-analysis behavior, and configuration decoding. Tasks include default analysis as well as focused recovery questions such as config extraction, decode-flow recovery, packing triage, and behavioral reconstruction. The corpus supports both breadth-oriented sweeps across many executables and depth-oriented studies on selected executables such as `config_decoder_test`.

6.3 Single Runs, Sweeps, and Repetitions

The harness supports both single-run evaluation and experiment sweeps. Sweeps are baseline-first and primarily one-variable-at-a-time, which shapes what kinds of causal claims the experiments can support.

A global baseline defines the default workflow configuration against which all variants are compared. In the maintained experiments, that baseline is not a deliberately weak strawman. It is a tool-complete staged static-analysis workflow with a light preparatory pass, deterministic presweeps, a balanced specialist worker mix, default prompt surfaces, and final reporting. Variant families then change one configuration surface at a time, and some families insert local family baselines where a fair comparison requires it. Repetitions support stronger claims about stability and significance. The evaluation artifacts include a repaired breadth sweep with one repetition per configuration and decoder-depth and final studies with three repetitions per configuration.

For reproducibility, each experiment family is defined as a self-contained launch preset. The exact preset identifiers, launch commands, and artifact locations are collected in Appendix A.6 so that the body chapter can focus on methodology rather than command syntax. The harness also separates preflight readiness artifacts from executed experiment outputs to keep the experiment record reserved for completed sweeps.

6.4 Broad Coverage and Final Held-Out Malware Evaluation

The evaluation design serves three distinct roles. Development and pilot runs on the authored benchmark are used to debug workflow behavior and identify promising interventions. Broader malware-facing coverage checks provide a realism test over manually supplied families. The final held-out evaluation run carries the main thesis-grade empirical claims.

The evaluation also includes a blind final-evaluation corpus. This corpus is intentionally evaluator-backed rather than a self-describing live-malware collection: the runtime sees only neutral sample aliases and task prompts, while the evaluator-facing reference set preserves family labels, source files, and scoring anchors. This separation matters both for safety and for scientific clarity. The broad-coverage pilot remains methodology-shaping, while the final held-out run carries the strongest empirical claims.

The held-out family set comprises Emotet, QakBot, TrickBot, WannaCry, and PlugX simulations, exposed to the runtime only as `sample1.exe` through `sample5.exe`. The final experiment runs one blind default-analysis task per sample across the baseline and a small set of development-informed comparison conditions. The comparison set is intentionally narrow: worker-subagent structure, worker-prompt shape, response scope, and architecture preset were varied because they proved more informative in development runs than obvious handicaps such as reduced tool access.

Evaluator-side reference files anchor the judge on sample-specific behaviors, evidence anchors, and overclaim boundaries without exposing family identity to the runtime agent.

A methodological caveat emerged only once the final held-out corpus was exercised at full scale. The final samples were materially more complex than the authored experimental benchmark, and that difference exposed a host-worker context-bloat failure mode that did not dominate the earlier development runs. In the final experiment, some Ghidra-heavy worker assignments exceeded the model input-context limit after large tool outputs and shared-state context were combined into a single worker request. This is important for interpreting the held-out results: part of the observed degradation is a workflow-scaling limitation, not only a reasoning-quality limitation. Chapter 7 therefore reports final performance alongside per-executable failure distributions rather than only aggregate score summaries.

6.5 Baselines and Experimental Variables

The harness can express several variable families, but the thesis experiments focus on those that survived development-stage screening: response scope, worker subagent profile, worker prompt shape, worker role-prompt mode, architecture preset, pipeline preset, and selected validator-backed pipeline follow-ups in the decoder-depth study. Appendix A.1 gives the complete mapping from variant labels to baseline-relative changes.

Prompt-shape and role-prompt changes are treated here as empirical variables rather than assumed improvements. That choice follows from both the archive-stage screening results and from external evidence that persona-style prompting does not reliably improve factual accuracy even when it changes response style [20].

6.6 Judging and Scoring

Run outputs are judged with a structured binary-analysis rubric and converted into continuous scores. The judging methodology matters because it defines how the experiments distinguish weak answers from unsupported or infrastructure-dominated failures.

Per-task judge outputs preserve structured feedback and an overall score. Legacy pass-or-fail thresholds still exist, but sweep analysis focuses on score deltas, success rates, error rates, timing summaries, and tool-use summaries rather than binary outcomes alone. The harness preserves analysis outcomes such as completed, analysis error, no result, judge error, and validator blocked. The final held-out evaluation scaffold also includes analyst-reviewed gold-label templates, which are intended to anchor evidence expectations and false-positive controls rather than merely family attribution.

6.7 Statistical Treatment and Comparison Strategy

The analysis in this thesis does not rely only on descriptive movement. The harness supports paired comparisons and limited significance analysis when repetitions or enough matched task pairs exist.

Where repeated runs exist, the harness supports replicate-level comparison. For matched sample-task pairs, it supports paired-task comparison. It reports score deltas, task success deltas, and significance outputs where the data support them. Throughout, the analysis maintains an explicit distinction between descriptive one-off sweep results and repeated experiments that can support stronger stability claims.

7

Results and Analysis

This chapter separates three kinds of evidence by evidentiary weight. First, the archived broad and decoder-depth studies are used as development-stage evidence: they helped narrow the final comparison set and exposed workflow design problems, but they are not the main basis for final claims. Second, the repaired final held-out experiment is used for the main performance comparisons because it restores complete coverage across all seven configurations. Third, the unrecovered final sweep is retained only as a robustness caveat, because it shows an infrastructure failure mode that the repaired aggregate would otherwise hide.

7.1 Development-Stage Archive Evidence

The archived broad and decoder-depth figures are still useful, but they should be read as methodology-shaping artifacts rather than as final effect estimates. Their main value is that they show which workflow changes were worth carrying into the held-out evaluation and which ones exposed design problems before the blind final comparison was launched. The broad pilot sample set is summarized in Table Table 1.1, and the variant labels used across the archive and final studies are summarized

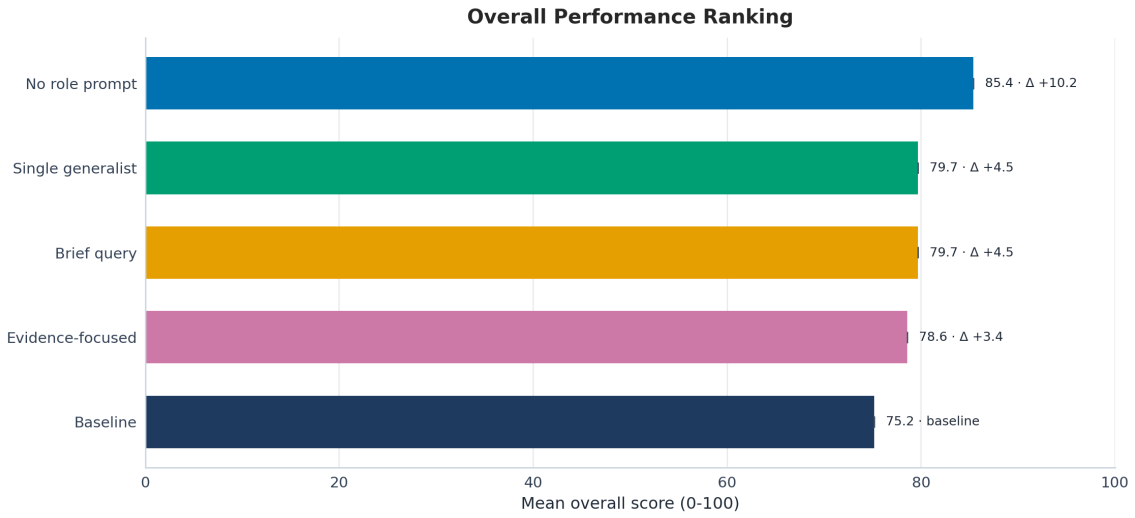

Figure 7.1: Archived broad-pilot ranking. This figure is used here as a descriptive development artifact: it shows which intervention families separated from the baseline strongly enough to justify follow-up study, but it is not treated as a thesis-grade estimate of general effectiveness.

in Table Table 1.4.

7.1.1 Broad Coverage Pilot

Figure Figure 7.1 supports two narrow conclusions. First, the pilot was sufficient to rule out some directions as uninteresting for the final comparison set. In particular, the final thesis sweep did not continue treating reduced tool access as a central variable, because that direction behaved more like an artificial handicap than an informative workflow comparison. Second, prompt-surface and worker-topology changes were large enough in the pilot to justify carrying them forward into deeper testing.

The broad pilot does *not* support a stronger claim than that. It is best read as a screening step over the authored benchmark. It helps explain why the final sweep focused on worker subagents, worker prompt shape, response scope, and architectural emphasis, but it does not by itself establish that any of those variables generalize to harder held-out malware families.

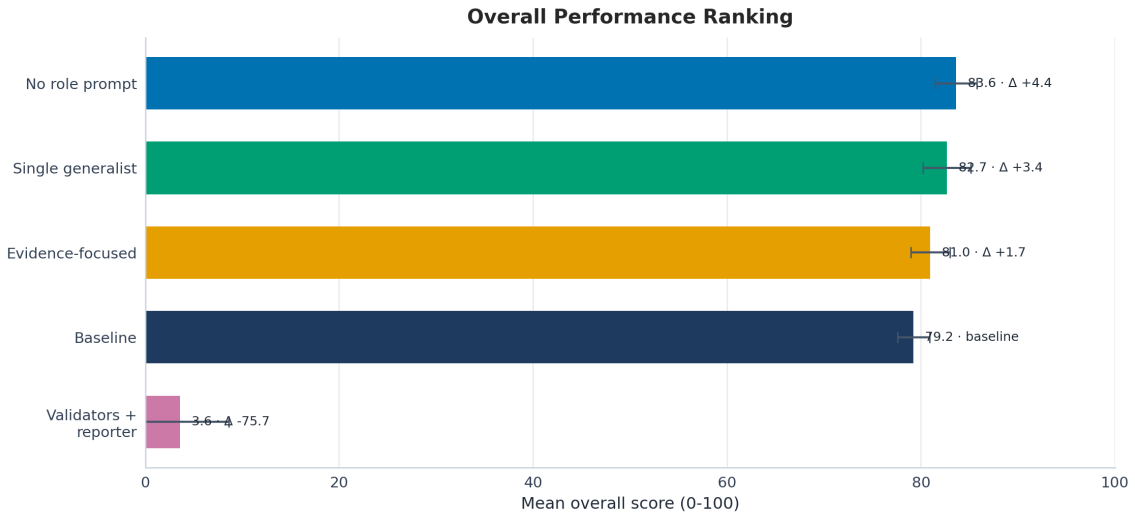

Figure 7.2: Archived decoder-depth ranking over the stripped and packed `config_decoder_test` variants. This case study is stronger than the broad pilot because it includes repeated runs, but it still remains a developmental case study rather than the final held-out evaluation.

7.1.2 Focused Decoder-Depth Case Study

The archived decoder-depth study supports a stronger but still limited interpretation. Compared with the broad pilot, it provides clearer evidence that workflow changes can matter differently on a deeper reverse-engineering task family than on easier overview-oriented tasks. Figure Figure 7.2 shows that worker prompt shape and worker topology can change performance on packed and stripped decoder-oriented cases rather than merely changing report style. Figure Figure 7.3 shows that not every additional layer helps. The validator-heavy pipeline behaves as a concrete negative case because its collapse is visible as a reliability and coverage failure rather than only a low score. The exact stripped and UPX-packed executables used in that case study are listed in Table Table 1.2, and the decoder-specific variant labels are defined in Table Table 1.4.

This case study therefore motivated two design choices for the held-out evaluation.

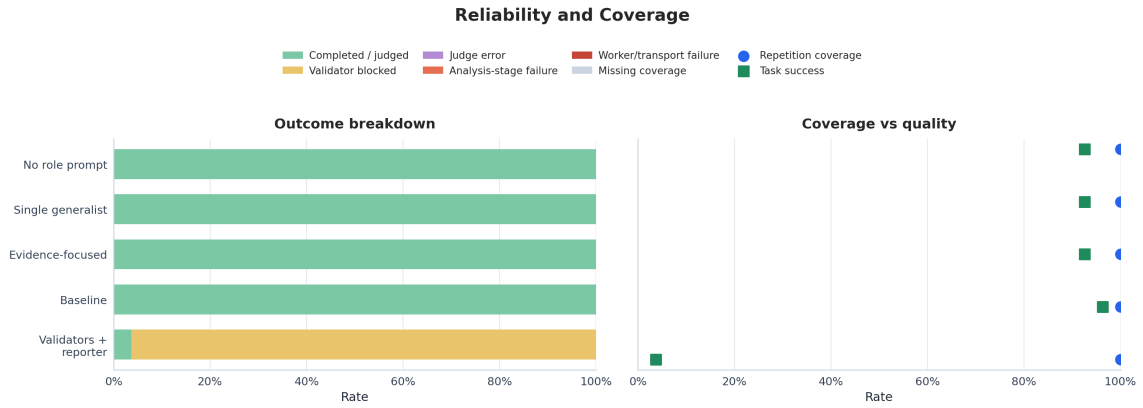

Figure 7.3: Archived decoder-depth reliability and coverage. This figure is especially important because it separates weak analytical output from a validator-dominated failure mode.

The final sweep kept worker-topology and prompt-shape comparisons, because the decoder-depth figures suggested they could matter on richer tasks. It did not elevate the validator-heavy pipeline into the final comparison set, because the development evidence suggested that its failure mode was structural rather than merely unlucky on one sample.

7.2 Final Held-Out Evaluation

Unless otherwise noted, performance comparisons in this section refer to the rebuilt repaired experiment, which preserves the benchmark scope and configuration set while folding in successful recovery results for tasks that initially failed for infrastructure reasons. (The exact experiment identifier and artifact provenance are documented in Appendix A.6.) The unrecovered sweep remains important and is discussed separately in Section 7.2.4, because it reveals a real robustness limitation of the workflow. The held-out sample aliases and evaluator-side technique summaries are listed in Table Table 1.3, and the configuration labels that appear in the figures are summarized in Table Table 1.4.

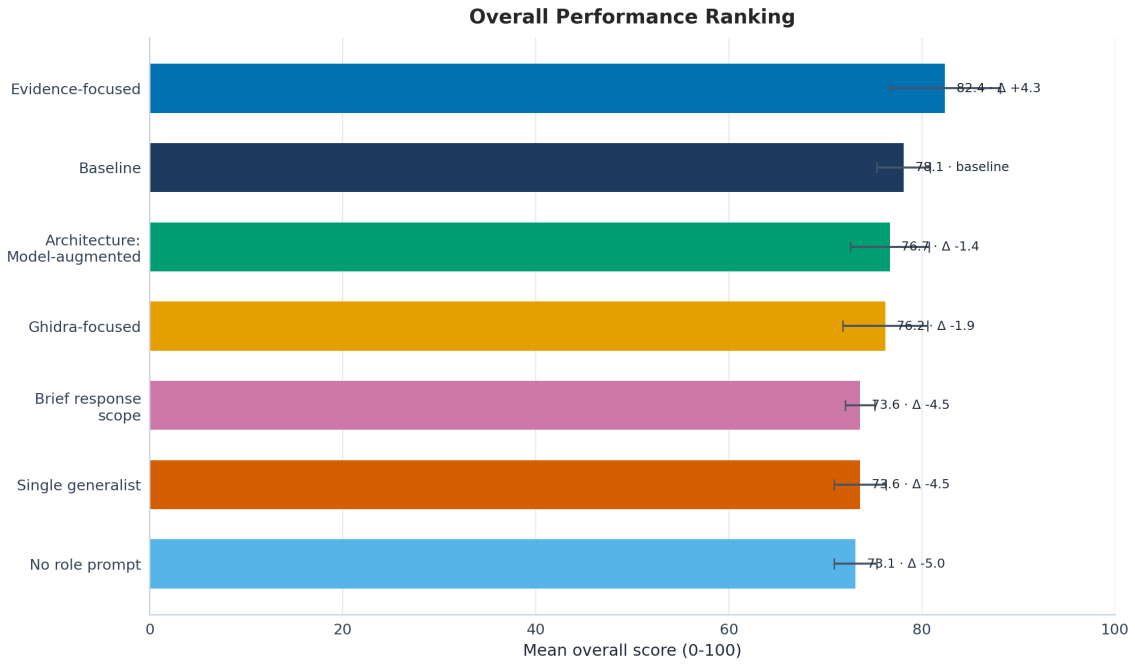

Figure 7.4: Repaired final held-out ranking across all seven configurations. This figure uses the rebuilt repaired experiment so that all configurations have full coverage. The highest descriptive mean score belongs to `worker_prompt_shape:artifact_focused`, but none of the overall shifts reached the thesis confidence target of $p \leqslant 0.05$.

7.2.1 Overall Performance After Recovery

Figures Figure 7.4 and Figure 7.5 support several direct findings. First, the final repaired comparison does *not* show a statistically significant overall winner at the thesis confidence target. The best descriptive overall mean score is produced by `worker_prompt_shape:artifact_focused` at `82.382`, compared with the baseline at `78.096`, but the replicate-level permutation test yields `p = 0.5`. Second, four of the six non-baseline variants underperform the baseline descriptively on overall mean score, and none of those degradations are statistically significant either. Third, the effect-size view shows why the chapter should stop short of a stronger claim: the descriptive winner moves in a positive direction, but the repaired final run still

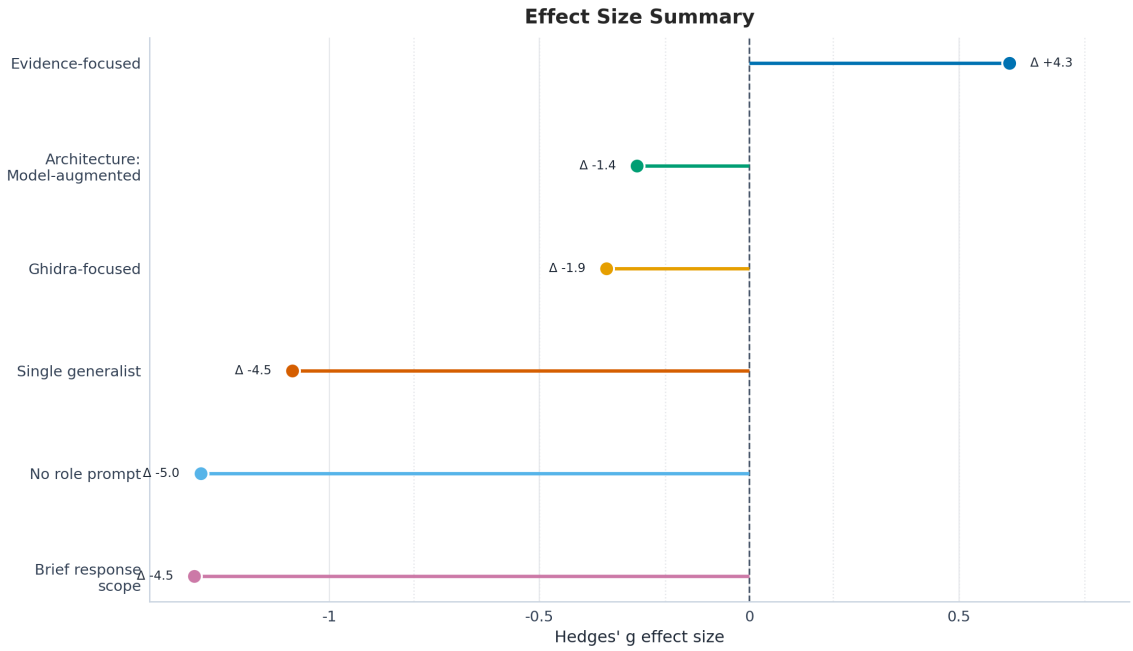

Figure 7.5: Effect-size summary for the repaired final held-out comparison relative to baseline. This figure matters because the repaired experiment supports descriptive ranking, but the inferential question is whether any overall shift is large and stable enough to treat as more than a sample-limited difference.

provides only three repetitions per configuration and none of the comparisons meets the thesis significance target.

These findings answer the first research question in a constrained way. The final repaired run does *not* justify claiming that any alternative workflow is superior overall on the held-out benchmark. It does, however, establish that the evidence-first worker-prompt overlay (`artifact_focused`) is the strongest descriptive candidate in the current design space, while the remaining alternatives either appear neutral-to-negative overall or depend heavily on which executable is being analyzed. That executable dependence motivates the finer-grained analysis in the next subsection.

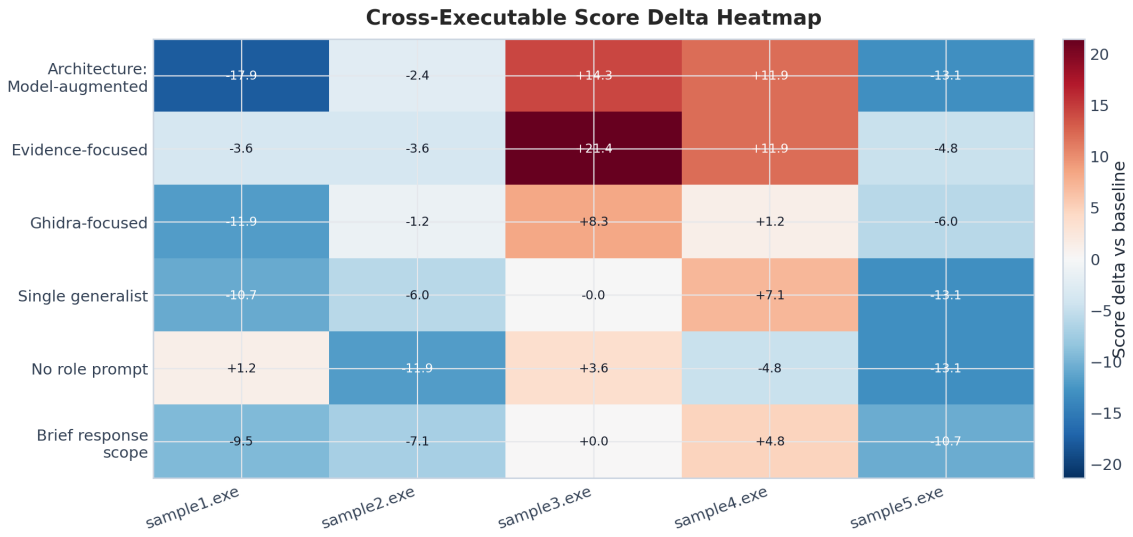

Figure 7.6: Cross-executable score-delta heatmap for the repaired final held-out experiment. Positive cells indicate improvement over the baseline on that executable and negative cells indicate degradation. This view matters because every non-baseline configuration in the repaired final run is marked as mixed and masked by average in the executable-consistency summary.

7.2.2 Executable-Level Heterogeneity

Figure Figure 7.6 shows why the overall mean is not enough. All six non-baseline configurations are heterogeneous across the five executables. In the executable-consistency summary, every non-baseline row is labeled `mixed`, and every one is flagged as `masked_by_average=true`. That is not a rhetorical caveat; it is a direct property of the repaired final dataset. The paged per-executable score panels generated from the same repaired experiment retain the repetition-level variance detail and are included in Appendix A.5, but the heatmap is the more legible thesis-scale summary and is therefore the figure kept in the main text.

The concrete pattern is easy to summarize. Baseline remains strongest on `sample2.exe` and `sample5.exe`. `worker_prompt_shape:empty` is the top descriptive performer only on `sample1.exe`, and even there the gain over baseline is modest at `+1.19`.

`worker_prompt_shape:artifact_focused` is strongest on `sample3.exe`, where it improves mean score by `+21.427` relative to baseline. `architecture_preset:model_augmented` ties for the best score on `sample4.exe` and improves by `+11.903` there, but it is also the worst-performing configuration on `sample1.exe` with a delta of `-17.856`. These are sample-dependent reversals, not minor fluctuations around the same answer. Table Table 1.3 is useful alongside this subsection because the alias names alone do not tell the reader which kinds of behaviors or analysis challenges each sample was meant to stress.

Two conclusions follow directly from the data. First, the final benchmark is heterogeneous enough that the configuration with the best global mean can still be a poor choice for specific executables, a finding that would be invisible in aggregate-only reporting. Second, the final repaired dataset supports *conditional* findings more confidently than *global* ones: some interventions help on certain executable profiles while hurting others. This answers the third research question more clearly than the first or second one. The held-out results show that executable-level heterogeneity is not a side note; it is central to how these workflows should be interpreted.

The next interpretive step is plausible but not fully proven: the executables that benefit from `artifact_focused` or `model_augmented` may be the ones where deeper control-flow reconstruction or evidence organization matters more than broad default coverage. The current experiment is suggestive on that point, but with only five executables it is better to treat that as a hypothesis for future stratified evaluation than as a settled causal explanation.

7.2.3 Efficiency and Repeated Tool Use

The repaired final run also supports a more operational analysis of efficiency. Figure Figure 7.7, together with the tool-redundancy summaries generated from the same repaired experiment, shows that repeated tool activity is not evenly distributed

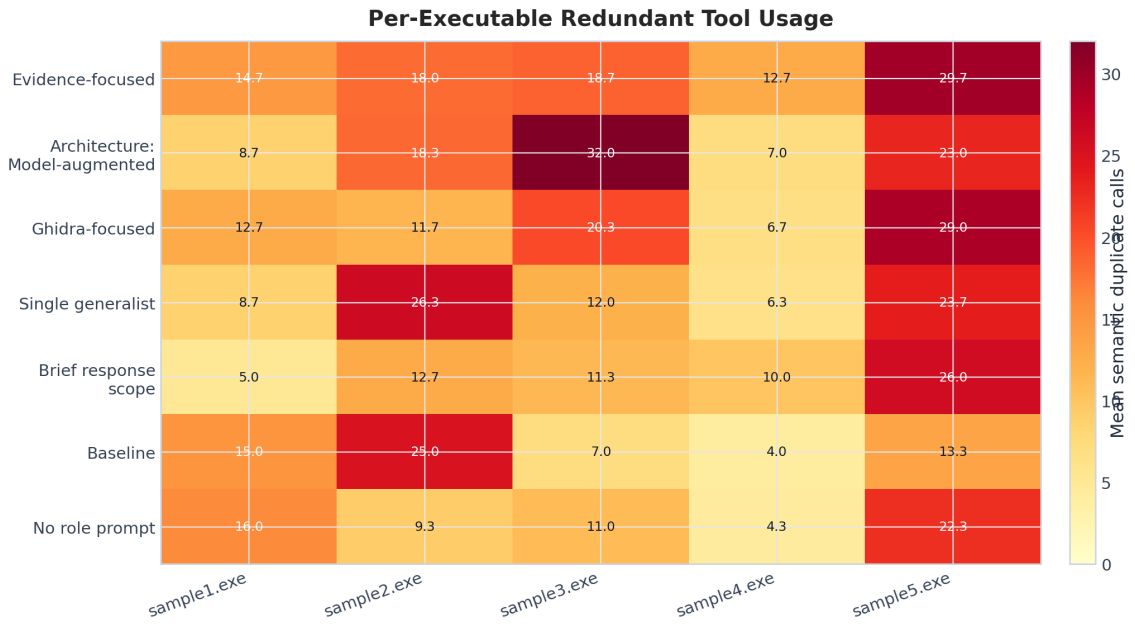

Figure 7.7: Per-executable semantic duplicate tool-call volume in the repaired final held-out experiment. The figure summarizes repeated tool activity within tasks, not proven waste. It is useful because it separates overall duplicate volume from the narrower same-source loops discussed in the text.

across executables or configurations. In total semantic duplicate volume, `sample5.exe` is the largest hotspot with `501` duplicate calls across the seven configurations, followed by `sample2.exe` with `364`. However, that broad volume should not be interpreted as pure waste. Much of the `sample5.exe` activity comes from cross-stage or cross-source overlap around `ghidra_program_info`, especially across preflight, deterministic presweeps, and worker stages.

The narrower same-source or likely-wasteful view tells a different story. When the analysis is restricted to repeats currently flagged as likely wasteful, `sample2.exe` becomes the largest hotspot with `26` likely-wasteful duplicate calls, followed by `sample3.exe` with `20`. Tool-family summaries show that this behavior is dominated by two patterns. The first is repeated `todo_read` state lookups, which account for `41` likely-wasteful duplicate calls overall. The second is a smaller but still visible set

of Ghidra rescans, especially `ghidra_program_info` (12) and `ghidra_imports` (11). Concrete hotspots include the model-augmented architecture on `sample3.exe`, where `todo_read` was called 22 times with 19 duplicates and 7 same-source repeats from `preflight.manager` and `reporter.manager`; and the single-generalist variant on `sample2.exe`, where `ghidra_program_info` was called 33 times with 30 duplicates across preflight, presweeps, and worker stages.

The complexity interpretation should remain cautious. Using baseline mean task wall-clock duration as a rough proxy for per-sample analysis difficulty, the repaired final data show only a moderate descriptive association with total semantic duplicate volume and little evidence of a clean relationship with likely-wasteful same-source loops. The longest baseline case, `sample5.exe`, does have the highest total duplicate volume, but `sample2.exe` has the highest likely-wasteful count despite a lower baseline wall-clock duration than `sample1.exe`, `sample3.exe`, or `sample5.exe`. The evidence therefore supports a limited claim: executables that are more ambiguous or artifact-heavy may induce more repeated tool activity overall, but the current data do not support a simple causal claim that executable complexity alone explains avoidable duplicate work.

Variant-level efficiency patterns are likewise mixed. The brief response-scope variant produces the highest total likely-wasteful duplicate count (19), while the baseline is lowest at 4. Yet repeated calls are not a complete explanation of runtime or score behavior. The evidence-focused worker-prompt variant is the only final variant whose mean wall-clock duration exceeds the baseline, but it is not the most duplication-heavy configuration. Conversely, the single-generalist and Ghidra-focused variants run faster than baseline while still producing more likely-wasteful duplicates. The actionable conclusion is therefore narrow: the data point to concrete opportunities for stage-level safeguards around repeated `todo_read` loops and for better reuse of Ghidra program or import context across preflight, presweeps, and worker stages,

but they do not justify treating duplicate counts as a standalone explanation for either score differences or runtime.

7.2.4 Robustness Caveat from the Original Held-Out Sweep

The repaired aggregate is the right artifact for comparing final scores, but the original unrecovered held-out sweep still matters as a methodological caveat. It exposed a real first-pass robustness problem concentrated on a subset of the harder samples, especially when Ghidra-heavy worker assignments accumulated too much shared context before completion. That matters because some one-shot failures were infrastructural rather than purely analytical, so the unrecovered sweep alone would understate the workflow's usable performance on the held-out benchmark.

That caveat should not dominate the results narrative, however. The main thesis question in this chapter is what can be learned from the completed final comparisons once recovery restores coverage. The unrecovered sweep is therefore best treated as context for interpreting those repaired comparisons, not as the central empirical result. The detailed outcome counts, per-executable failure figures, and assignment-failure breakdown are moved to Appendix A.4, and Chapter 8 discusses what they do and do not imply about robustness.

7.3 Discussion and Claim Boundaries

7.3.1 Findings Directly Supported by the Data

The repaired final experiment supports five direct findings. First, no non-baseline configuration achieved a statistically significant overall score improvement at the thesis confidence target. Second, `worker_prompt_shape:artifact_focused` is the strongest descriptive overall candidate in the repaired final comparison, but that claim is descriptive rather than confirmatory. Third, executable-level behavior is strongly heterogeneous: every non-baseline configuration is mixed across the five exe-

cutables, and every one is marked as masked by average in the executable-consistency summary. Fourth, repeated tool activity is concentrated on a subset of executables and tools rather than being evenly distributed across the benchmark; the largest likely-wasteful hotspots come from `todo_read` loops and a smaller set of Ghidra rescans. Fifth, the original unrecovered held-out sweep exposed a real workflow robustness problem dominated by host-worker context overflow on Ghidra-heavy assignments.

7.3.2 Plausible Interpretations

The results also suggest, but do not prove, several interpretations. The first is that evidence-first prompt shaping may help most on executables where the bottleneck is organizing and grounding many competing artifacts rather than simply producing a concise report. The second is that architecture-level changes such as `model_augmented` may be acting as conditional strategies rather than universal improvements: they help on some executables and clearly hurt on others. The third is that the held-out benchmark is stressing two capabilities at once, namely reverse-engineering quality and orchestration robustness. The failure analysis makes it plausible that some of the apparent performance difference between configurations is mediated by how well they keep worker contexts tractable, not only by how well they reason after the context is assembled. The efficiency summaries suggest a related operational hypothesis: some repeated work is likely caused by missing stage-to-stage reuse or state-loop controls rather than by analytical necessity alone.

7.3.3 Open Uncertainties and Measurement Gaps

Several uncertainties remain open. The final comparison still covers only five held-out executables and three repaired repetitions per configuration, so the executable-level patterns should be treated as strong case evidence rather than as a stable estimate

of population-level behavior. The significance tables reinforce that caution: none of the overall score shifts reached the thesis significance threshold, and no multiple-comparison correction is applied in the exploratory task-level tables. In addition, the repaired experiment necessarily combines original runs with targeted recovery runs. That is appropriate for obtaining full benchmark coverage, but it means the final aggregate should be read as the best-available completed comparison under the evaluated workflow rather than as a pristine one-shot sweep.

The tool-usage summaries are also useful but limited. They identify repeated-call hotspots and they now distinguish same-source repeats from cross-source overlap within a task. However, they still do not attach every tool call to a specific work item inside a reused worker slot. That means the current instrumentation can support statements such as "this task and source produced many repeated calls" or "same-source repetition exists here," but it cannot yet prove that each repeated call was truly unnecessary. The same repaired final study also uses a single *default analysis* task per held-out executable and keeps the `repo_default` model profile fixed across all seven configurations. As a result, the current efficiency analysis cannot cleanly separate task-type effects from executable effects or attribute redundancy differences to model-family changes. Those measurement limits are discussed more fully in Chapter 8.

Taken together, the repaired final results support three evidence-backed conclusions. First, no single configuration can be claimed as globally best with statistical confidence at the thesis significance target. Second, workflow choices matter materially at the executable level: `worker_prompt_shape:artifact_focused` is the strongest descriptive overall candidate and produces the largest positive delta on `sample3.exe` (+21.4 over baseline), but the same configuration does not dominate uniformly. Third, executable-level heterogeneity is large enough that average-only reporting would be misleading: every non-baseline configuration is flagged as

`masked_by_average` in the consistency summary. These findings frame agentic reverse engineering as a workflow-selection and robustness challenge rather than a prompt-tuning problem with a single optimal answer.

8

Limitations and Threats to Validity

This chapter states the main boundaries on the claims made in this thesis. Some of these limits come from the benchmark itself, some from the model and workflow choices, and some from the measurement and reporting stack. Making those limits explicit is important because the workflow combines nontrivial orchestration, tool integration, structured judging, and repaired experiment outputs.

8.1 Benchmark Scope and External Validity

The final held-out evaluation is stronger than the earlier development studies, but it remains a small benchmark. The thesis-grade comparison covers five held-out executables, one locked default-analysis task per executable, and three repaired repetitions per configuration. That scope is sufficient to demonstrate that executable-level heterogeneity matters, but it is not sufficient to support a broad claim that any single workflow is generally best for malware reverse engineering.

The authored development benchmark introduces a different limitation. It is useful for controlled workflow debugging and ablation design, but it is not a substitute for a large external malware corpus. The archived broad and decoder-depth studies

therefore function as methodological evidence and case-study evidence, not as the final basis for generalization.

8.2 Statistical Power and Repetition Count

The repaired final experiment improves coverage, but it does not remove the small-sample problem. Each configuration has only three repaired repetitions, and the final evaluation includes only five executables. That directly limits statistical power. In the final repaired experiment, no overall score comparison reached the thesis significance target of $p \leqslant 0.05$. This means the final chapter can support descriptive comparisons and executable-level case analysis more confidently than strong confirmatory claims about superiority.

The significance outputs should therefore be read as guardrails rather than as the main argument. They help prevent overclaiming when an observed mean difference looks appealing, but they do not overcome the basic limitation that both the replicate count and the executable count remain modest. Additional executables, more repetitions, or both would be needed before making stronger population-level claims.

8.3 Model Scope

The empirical results in this thesis are tied to the specific model budget and workflow choices used in the evaluated runs. In practice, much of the execution used mini- and nano-class models rather than larger frontier models. That matters in at least two ways.

First, some of the observed failure modes are plausibly model-class sensitive. The context-window and request-budget problems that appeared in the final run may behave differently with larger models, different context windows, or different provider-side limits. Second, positive or negative results under mini/nano models do not automatically transfer to larger models. A workflow that looks brittle here

may become more robust with a larger context window, while a workflow that looks cost-effective here may lose that advantage if evaluated only with a more capable but more expensive model.

For that reason, the current findings are best read as evidence about *this* workflow under *this* budget-conscious model regime, not as a definitive statement about all agentic reverse-engineering systems.

8.4 Recovery and One-Shot Robustness

The final performance chapter intentionally uses the repaired experiment for score comparison and the unrecovered sweep for robustness analysis. That is the most honest way to preserve both pieces of evidence, but it also introduces a limitation. The repaired experiment is not the same thing as a clean one-shot evaluation in which every task completed successfully on the first attempt.

This matters because recovery is doing real work. In the held-out evaluation, the unrecovered sweep exposed infrastructure-heavy failure modes, especially host-worker context overflows on Ghidra-heavy assignments. Later patches and recovery runs improved completeness materially. That makes the repaired experiment the right artifact for final like-for-like comparison, but it also means that the repaired tables should be interpreted as "best available completed comparison under the evaluated workflow" rather than as evidence that the workflow was robust on its first pass.

8.5 Tool-Call Redundancy Measurement

The redundancy reporting added during this work is informative, but it does not justify the strongest possible claims about unnecessary tool use. The current implementation records tool calls per task and preserves a `source` label that usually maps to a stage manager or worker slot. In other words, the current summaries do *not*

first pool together calls from unrelated tasks before counting redundancy. That is enough to distinguish several important cases:

It can distinguish repeated calls within one sample-task run from calls originating in different tasks, same-source repetition from cross-source overlap within the same task, and broad within-task hotspots such as repeated program-info queries or repeated `todo_read` state lookups.

However, the current logs still do not attach every tool call to a specific work item or objective inside a reused worker slot. As a result, the reporting can reliably say that a given task and source produced repeated calls, but it cannot always say whether those calls were truly excess. Some repeated calls may represent legitimate cross-stage refreshes, parallel workers collecting shared context within one task, or one worker slot being reused for multiple different objectives inside the same task.

The consequence is important for interpretation. The redundancy figures and tables are good for identifying hotspots, likely same-source loops, and areas where better memory or caching may help. They are *not* strong enough to prove that every repeated call counted in the summaries was wasteful. The same caution applies to complexity-based interpretations: the final repaired run shows that longer-running executables tend to have somewhat more total repeated activity, but that relationship is weak for the narrower same-source or likely-wasteful subset and does not behave monotonically across the five held-out samples. Stronger instrumentation, especially per-call work-item attribution and a larger benchmark with more than one held-out task type, would be needed before making a firmer claim about exact avoidable overhead.

Another boundary on this analysis is model scope. In the final held-out comparison, all seven configurations use the same `repo_default` model profile and the same `auto_triage` pipeline family; the architecture variants change worker mix, not provider or model family. That is useful because it keeps the main efficiency com-

parisons focused on workflow differences, but it also means the current thesis cannot support a separate claim about model-specific redundancy behavior.

8.6 Judging and Measurement Proxies

Judge scores remain a proxy for analyst usefulness, not a direct substitute for expert human review. The rubric is structured and useful, but it still compresses a complicated reverse-engineering outcome into a set of scalar dimensions and one overall score. That creates two related risks.

Some runs may benefit from better organization or cleaner evidence presentation without necessarily achieving deeper mechanistic understanding. Conversely, a partially correct but poorly framed answer may score worse than an analyst would judge it in an interactive workflow where clarification is possible.

The thesis reduces this risk by pairing score reporting with failure-mode reporting, per-executable comparisons, and preserved task artifacts. Even so, the judge should be treated as a consistent evaluation instrument rather than as a perfect oracle.

8.7 Operational and Ethical Considerations

Finally, the workflow is inherently dual-use. Malware-oriented reverse-engineering tooling can assist defensive analysis, but it can also accelerate unsafe understanding of malicious capabilities. The system mitigates that risk through local execution, evaluator-side corpus control, and explicit workflow logging, but those mitigations do not eliminate the dual-use concern. Any broader operational deployment would require stronger containment, access control, and review procedures than those needed for a thesis evaluation environment.

9

Conclusion and Future Work

This chapter restates the core problem, summarizes the main findings, and identifies the next research steps.

9.1 Conclusion

This thesis studied whether reverse engineering and malware analysis benefit from a workflow that treats tool use, staged reasoning, review, and result provenance as first-class concerns rather than as incidental prompt-engineering details. The system contribution is a modular agentic workflow that separates preparation, planning, specialist evidence gathering, optional validation, and final reporting, together with an evaluation harness that can run baseline-first sweeps, preserve per-task artifacts, recover failed cases, and regenerate comparison outputs reproducibly.

The empirical contribution is more specific than a claim that one workflow is simply best. The development-stage broad and decoder-depth studies were useful for screening the design space and for identifying which variables were worth carrying into the held-out comparison. They showed that prompt shape and worker topology were more informative than obvious handicaps such as reducing tool access, and they

exposed structural failure modes such as validator-heavy collapse before the blind final run was launched.

The repaired held-out evaluation provides the strongest evidence in the thesis. No non-baseline configuration achieved a statistically significant overall improvement over the baseline at the thesis confidence target ($p \leqslant 0.05$). The strongest descriptive overall candidate was `worker_prompt_shape:artifact_focused`, which produced its largest gain on `sample3.exe` (+21.4 over baseline). However, the final comparison also showed strong executable-level heterogeneity: the same configuration could help on one held-out sample and hurt on another, and every non-baseline row was flagged as masked by average in the consistency summary. The clearest evidence-backed takeaway is therefore not that the thesis discovered a universally best configuration, but that workflow choices matter conditionally and that average-only reporting hides meaningful sample-specific reversals.

The held-out run also showed that orchestration robustness is itself part of the research problem. The unrecovered sweep exposed context-bloat and request-budget failures that were not dominant in the smaller development benchmarks. Those failures do not erase the repaired comparisons, but they do show that agentic reverse engineering must be evaluated as both a reasoning problem and a workflow-engineering problem. A system that can reason well in principle is still limited if it cannot keep tool outputs, worker contexts, and recovery paths operationally tractable on harder binaries. The efficiency summaries reinforce that point: repeated tool activity is concentrated on a subset of executables and is dominated by specific patterns such as `todo_read` loops and selected Ghidra rescans, which suggests that workflow overhead is not purely a model-quality issue.

Taken together, these results answer the thesis research questions in a bounded way. Relative to the maintained baseline, the experiments do not support a claim of global superiority for any tested alternative workflow. They do support the claim

that workflow variables such as worker prompt shape and architecture emphasis can materially change outcomes on specific executables. They also show that executable-level heterogeneity is large enough to make aggregate averages incomplete on their own, and that workflow robustness remains part of the empirical problem rather than a secondary implementation nuisance.

9.2 Future Work

Several next steps follow directly from these findings.

Broader benchmarks. Additional held-out executables, more repetitions, and more than one locked task per family would clarify whether the executable-level heterogeneity observed here is a stable pattern or a sample-limited artifact.

Better instrumentation. Per-call work-item attribution, stronger caching or memory controls, and richer robustness accounting would make it possible to separate analytical weakness from orchestration overhead more cleanly than the current summaries allow. The repaired final run suggests two especially practical targets: stage-level guards around repeated `todo_read` loops and better reuse of Ghidra program or import context across preflight, presweeps, and worker stages. Trace-derived workflow compression through meta-tools is one concrete example of the kind of optimization that could reduce repeated tool sequences without changing the underlying model [19].

Model-class generalization. The system should be evaluated beyond the budget-conscious mini/nano regime used in the current runs. Larger models, different provider limits, or different context windows may change both the benefit of staged orchestration and the failure modes that dominate harder samples.

Dynamic analysis. Execution-oriented analysis would require a more substantive implementation than the current system provides. The architecture reserves metadata for distinguishing static from dynamic tool domains, but the evaluated system does not include a real dynamic-analysis MCP lane. A future study could add controlled execution, runtime tracing, or sandbox evidence collection and then evaluate when that extra evidence genuinely improves grounded analysis rather than only adding complexity.

More broadly, the thesis suggests a shift in emphasis for future research on agentic reverse engineering. The most useful question may not be "which single workflow is best?" but rather "how should a workflow adapt to different executable profiles while remaining grounded, auditable, and operationally reliable?" That is a harder problem than prompt tuning alone, but it is also the more realistic one for reverse engineering as a long-term analyst-assistance direction.

Bibliography

[1] P. Kunwar, K. Aryal, M. Gupta, M. Abdelsalam, and E. Bertino, "SoK: Leveraging transformers for malware analysis," arXiv preprint arXiv:2405.17190, 2024. [Online]. Available: https://arxiv.org/abs/2405.17190

[2] J. Mattei, M. McLaughlin, S. Katcher, and D. Votipka, "A qualitative evaluation of reverse engineering tool usability," in *Proceedings of the 38th Annual Computer Security Applications Conference*, Austin, TX, USA, Dec. 2022, pp. 619–631.

[3] K. Yakdan, S. Dechand, E. Gerhards-Padilla, and M. Smith, "Helping johnny to analyze malware: A usability-optimized decompiler and malware analysis user study," in *2016 IEEE Symposium on Security and Privacy (SP)*, San Jose, CA, USA, May 2016, pp. 158–177.

[4] Anthropic, "Disrupting the first reported AI-orchestrated cyber espionage campaign," Anthropic, Full Report, Nov. 2025. [Online]. Available: https://assets.anthropic.com/m/ec212e6566a0d47/original/Disrupting-the-first-reported-AI-orchestrated-cyber-espionage-campaign.pdf

[5] Federal Bureau of Investigation, "2025 IC3 Annual Report," Internet Crime Complaint Center, Annual Report, 2025. [Online]. Available: https://www.ic3.gov/AnnualReport/Reports/2025_IC3Report.pdf

[6] T. Raitsis, Y. Elgazari, G. E. Toibin, Y. Lurie, S. Mark, and O. Margalit, "Code obfuscation: A comprehensive approach to detection, classification, and ethical challenges," *Algorithms*, vol. 18, no. 2, p. 54, Jan. 2025.

[7] "Lauriewired/ghidramcp: MCP server for ghidra," https://github.com/LaurieWired/GhidraMCP, 2025, accessed: Dec. 12, 2025.

[8] "Plugin," https://ghidra.re/ghidra_docs/api/ghidra/framework/plugintool/Plugin.html, 2025, accessed: Dec. 12, 2025.

[9] D. Xie, Z. Zhang, N. Jiang, X. Xu, L. Tan, and X. Zhang, "Resym: Harnessing LLMs to recover variable and data structure symbols from stripped binaries," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, Salt Lake City, UT, USA, Dec. 2024, pp. 4554–4568.

[10] L. E. Erdogan, N. Lee, S. Kim, S. Moon, H. Furuta, G. Anumanchipalli, K. Keutzer, and A. Gholami, "Plan-and-act: Improving planning of agents for long-horizon tasks," 2025. [Online]. Available: https://arxiv.org/abs/2503.09572

[11] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[12] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[13] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," 2024. [Online]. Available: https://arxiv.org/abs/2305.11738

[14] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "Jtrans: jump-aware transformer for binary code similarity detection," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, Virtual, South Korea, Jul. 2022, pp. 1–13.

[15] X. Shang, G. Chen, S. Cheng, B. Wu, L. Hu, G. Li, W. Zhang, and N. Yu, "Binmetric: A comprehensive binary analysis benchmark for large language models," 2025. [Online]. Available: https://arxiv.org/abs/2505.07360

[16] L. Hu, X. Shang, J. Shi, S. Cheng, J. Zhang, G. Li, Z. Yang, W. Zhang, and D. Lo, "Can LLMs deobfuscate binary code? a systematic analysis of large language models into pseudocode deobfuscation," 2026. [Online]. Available: https://arxiv.org/abs/2604.08083

[17] jtang613, "jtang613/ghidrassist," https://github.com/jtang613/GhidrAssist, 2025, java. Accessed: Dec. 12, 2025.

[18] H. Zhou, X. Wan, R. Sun, H. Palangi, S. Iqbal, I. Vulić, A. Korhonen, and S. Ö. Arık, "Multi-agent design: Optimizing agents with better prompts and topologies," 2026. [Online]. Available: https://arxiv.org/abs/2502.02533

[19] S. Abuzakuk, A.-M. Kermarrec, R. Sharma, R. M. Veski, and M. de Vos, "Optimizing agentic workflows using meta-tools," 2026. [Online]. Available: https://arxiv.org/abs/2601.22037

[20] S. Basil, I. Shapiro, D. Shapiro, E. R. Mollick, L. Mollick, and L. Meincke, "Prompting science report 4: Playing pretend: Expert personas don't improve factual accuracy," SSRN Electronic Journal, Dec. 2025, date written: Dec. 5, 2025. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5879722

[21] E. J. Schwartz, C. F. Cohen, M. Duggan, J. Gennari, J. S. Havrilla, and C. Hines, "Using logic programming to recover C++ classes and methods from compiled executables," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, Toronto, Canada, Oct. 2018, pp. 426–441.

[22] "Hashdb," https://hashdb.openanalysis.net/, 2025, accessed: Dec. 12, 2025.

[23] "horsicq/detect-it-easy: Program for determining types of files for windows, linux and macOS," https://github.com/horsicq/Detect-It-Easy, 2025, accessed: Dec. 12, 2025.

[24] "Malwarebazaar — API," https://bazaar.abuse.ch/api/, 2025, accessed: Dec. 12, 2025.

Appendix A

Supplementary Evaluation Material

This appendix collects the concrete sample, prompt, configuration, and launch details that support the main results chapters. The goal is traceability: when the main text refers to a sample family, a variant label, or a maintained preset, the corresponding evaluator-side details should be easy to find here without interrupting the flow of the main argument.

A.1 Sample and Variant Traceability

The thesis uses three evaluation groups with different roles. The broad pilot screens a wider medium-and-hard authored corpus for promising workflow dimensions. The depth-focused studies reuse a smaller decoder-oriented slice to stress stripped and packed recovery tasks. The final held-out evaluation uses five opaque aliases backed by evaluator-side references and is the strongest basis for the main empirical claims. Tables Table 1.1–Table 1.3 map each executable group to the implemented methods or intended analysis challenge it represents, and Table Table 1.4 maps the maintained variant labels back to their baseline-relative changes.

A.1.1 Broad Coverage Pilot Executables

The broad pilot preset `coverage_broad_r1_60usd` runs over the medium-and-hard authored slice of the experimental corpus. Its purpose is not to emulate a single malware family but to cover a range of reverse-engineering challenges including decoy-heavy naming, stripping, hashing-based dispatch, packing, embedded payload parsing, and anti-analysis handling.

Table 1.1: Executable set used by the maintained broad-coverage pilot. The third column summarizes the main implemented techniques or intended analysis challenge for each sample.

Sample	**Diff.**	**Major implemented methods or intended challenge**
`maintenance_orchestrator_test.exe`	hard	Misleading function names, decoy strings, dormant decoded notices, and a live/dead opcode dispatcher wrapped around a deterministic local maintenance-report workflow; primarily tests false-positive control and live-path recovery.
`maintenance_orchestrator_test_stripped.exe`	hard	Stripped version of the maintenance-orchestrator case; removes symbol-name shortcuts and forces the same benign workflow to be recovered from code and data flow alone.
`callback_dispatch_test.exe`	medium	Function-pointer callback table plus struct-based command descriptors and indirect calls; tests recovery of a callback-driven state machine from data structure and dispatch evidence.
`config_decoder_test.exe`	medium	Single-byte XOR configuration blob, key/value parser, stack-buffer decode path, and validation branching; serves as the clean decoder baseline for later depth-focused studies.
`signal_router_report_test.exe`	medium	Threat-looking helper names and decoy labels hide local signal-routing and report-generation behavior; tests whether the workflow follows real data flow instead of surface wording.
`stack_notice_scheduler_test.exe`	medium	Stack-built decoy strings, dormant XOR-decoded notices, decoded minute offsets, and insertion-sort schedule ordering; tests cautious interpretation of FLOSS-style recovered strings and scheduling logic.

Sample	Diff.	Major implemented methods or intended challenge
multilayer_encode_test.exe	hard	Layered ROL+XOR encoding around URLs, commands, per-entry metadata, and a magic header; tests multi-stage decode reasoning rather than one-pass string recovery.
hash_dispatch_test.exe	hard	DJB2 hash algorithm, hash-to-handler dispatch table, stack-built command strings, and dead hash entries; tests concrete hash-dispatch reconstruction.
hash_dispatch_test_stripped.exe	hard	Stripped variant of the hash-dispatch case; preserves the dispatcher logic while removing preserved names, so behavior must be recovered from constants and control flow.
embedded_payload_test.exe	hard	Embedded blob with a structured header, rolling checksum, per-record XOR keys, and staged extraction logic; tests payload parsing and staged extraction reasoning.
embedded_payload_test_upx.exe	hard	UPX-packed version of the embedded-payload case; adds packer recognition and unpack-aware reasoning before the same staged extraction problem.
branch_weave_snapshot_test.exe	hard	Encoded operation stream and dispatcher hidden behind misleading names, weak live strings, and inert decoded notices; tests control-flow-led behavior recovery when strings are poor evidence.
branch_weave_snapshot_test_stripped.exe	hard	Stripped version of the branch-weave case; pushes the same grid-building and report-generation logic into a symbol-poor setting.
anti_analysis_suite_test.exe	hard	Anti-debug APIs, timing probes, environment fingerprinting, rotating-key XOR config, dead code, and misleading names; tests anti-analysis identification without inventing unsupported payload claims.

A.1.2 Focused Decoder-Depth Executables

The repeated decoder-depth preset `sweep_decoder_depth_r3` and the follow-up preset `sweep_decoder_depth_followups_r3` both reuse the same two executables and the same deeper task slate: `default_analysis`, `config_value_recovery`, `decode_parser_flow_recove` `parser_validation_audit`, and `packing_decoder_triage`. This narrower group is intended to stress symbol stripping, parser reconstruction, and packer-aware recovery more heavily than the broad pilot does.

A.1.3 Final Held-Out Executables

The final preset `thesis_final_eval_r3_100usd` runs over five opaque aliases under `Testing/sources/final\_round/`. The runtime only sees the aliases in the first column. The evaluator-side family labels and behavior summaries remain reference-only metadata and are included here so the results chapter can discuss held-out sample heterogeneity precisely without leaking those labels into the prompt surface itself.

A.1.4 Maintained Variant and Configuration Labels

Unless a row below says otherwise, the baseline refers to the locked configuration used across the thesis experiments: `pipeline=auto_triage`, `architecture=balanced`, `response_scope=default`, `analysis_hint=default`, `subagent_profile=default`, `worker_prompt_shape=default`, `tool_profile=full`, and `model_profile=repo_default`. The table below summarizes the maintained variant labels that appear in the archive graphs and final held-out results.

Table 1.4: Maintained configuration variants referenced throughout the thesis. Each row describes what changes relative to the locked baseline and where that variant appears in the evaluation program.

Variant label	**Used in**	**What changes relative to baseline**	**Dimension**
`baseline`	broad, depth, final	No variant override. Serves as the common reference point for all reported deltas and rankings.	reference
`response_scope:brief`	broad, final	Changes only the response wrapper so the requested final report is more concise and evidence-focused; it does *not* inject new sample-specific hints into the task itself.	prompt framing
`worker_subagents:single_generalist`	broad, depth, final	Replaces the default worker-specialist mix with a single generalist worker slot while leaving the rest of the pipeline unchanged.	topology

Variant label	Used in	What changes relative to baseline	Dimension
`worker_prompt_shape:artifact_focused`	`artifact_focused`, `depth` follow-up, final	Keeps the default worker-role structure but adds an evidence-first worker persona overlay intended to prioritize concrete artifacts and stronger grounding.	prompting
`worker_prompt_shape:empty`	`empty`, `depth` follow-up, final	Blanks the worker-role prompt while keeping the rest of the workflow fixed, making it a prompt-surface ablation rather than an architecture change.	prompting
`pipeline_preset:validators_reporter`	`depth` follow-up only	Replaces the baseline `auto_triage` path with a planner-workers-validators-reporter pipeline so validator review becomes an explicit stage.	pipeline
`architecture_preset:ghidra_focused`	`ghidra_focused`	Shifts the architecture toward a more Ghidra-centered worker mix and evidence-gathering pattern while preserving the same held-out tasks and tool profile.	architecture
`architecture_preset:model_augmented`	`model_augmented`	Shifts the architecture toward more model-side synthesis and augmentation while holding the benchmark scope, prompt surface, and tool profile constant.	architecture

A.2 Judge and Prompt Artifacts

The judge rubric is defined in `Testing/config/binary_judge_rubric.json`. It scores six dimensions: `evidence_grounding`, `specificity`, `technique_coverage`, `false_claim_control`, `task_alignment`, and the down-weighted `report_conciseness`. The first five dimensions are each scored from 0 to 5, while conciseness is scored from 0 to 3. Raw totals are normalized to a 0–100 score by dividing by the rubric maximum of `28`. A run is treated as passing only if the normalized overall score is at least `70`, `false_claim_control` is at least `3`, and `evidence_grounding` is at least `3`. This structure is intended to prevent short but weak reports, or confident but unsupported reports, from appearing successful.

The base held-out task prompt is deliberately broad and blind:

```
Analyze this executable as an unknown sample.  Recover the main
behaviors, concrete evidence anchors, anti-analysis or packing
indicators, and any persistence, networking, or process-execution
behavior while clearly separating supported findings from uncertainty.
```

The response-scope wrappers in `Testing/config/response\_scope\_variants.json` do not change the underlying task itself. Instead they wrap that base query with a prefix and suffix that alter how concise or expansive the requested answer should be. For example, the maintained `brief` variant adds the prefix "Provide a concise, evidence-grounded reverse-engineering answer" and the suffix "Keep the report focused on the most important findings, the strongest supporting evidence, and the main unknowns." The analysis-hint wrappers in `Testing/config/analysis\_hint\_variants.json` are separate and are currently reserved for future pilot studies. Those wrappers add verify-first investigation leads such as packing, encoded configuration, hashing-based dispatch, or embedded payloads without asserting that such behaviors are present. Table Table 1.4 summarizes which of those maintained wrapper variants are actually exercised in the thesis experiments.

The stage-level prompt surface is split across three files under `multi\_agent\_wf/workflow\_config/`: `stage\_manager\_prompts.json`, `agent\_archetype\_prompts.json`, and `stage\_output\_contracts.json`. The first defines the coordinator-level rules for each pipeline stage, the second defines specialist overlays such as `ghidra_analyst`, `control_flow_analyst`, or `string_analyst`, and the third defines the machine-readable blocks or evidence-bundle contracts that later stages must consume.

A.3 Configuration and Experiment Provenance

The workflow-level configuration surface is concentrated in `multi\_agent\_wf/workflow\_config/`. The most important files are `architecture\_presets.json` for manager and worker topologies, `pipeline\_presets.json` for stage sequences, `agent\`

_archetype_specs.json and agent_archetype_prompts.json for specialist roles and tool-domain mapping, stage_manager_prompts.json for coordinator instructions, stage_output_contracts.json for machine-readable stage outputs, and worker_persona_profiles.json for worker prompt-shape overlays. The lower-level base prompt scaffolding is defined in base_prompts.json.

The evaluation-side configuration surface lives under Testing/config/. The most important files are presets/*.json for runnable named presets, experiment_sweeps.json for the generic direct-sweep definition, tool_profiles.json for MCP availability profiles, response_scope_variants.json and analysis_hint_variants.json for prompt wrappers, binary_judge_rubric.json and prompts/binary_judge_prompt.md for judging, and budget_guardrails.json plus model_cost_profiles.json for launch-time cost and safety constraints. Corpus definitions live alongside the sample sets themselves, most importantly Testing/sources/experimental/sample_manifest.json and Testing/sources/final_round/sample_manifest.json.

The preset files correspond directly to the main experiment families discussed in this thesis: coverage_broad_r1_60usd for the breadth-oriented pilot, sweep_decoder_depth_r3 and sweep_decoder_depth_followups_r3 for the decoder-depth studies, and thesis_final_eval_r3_10 for the blind held-out final evaluation. Each preset is self-contained and names its corpus, selected samples or tasks, repetition count, baseline configuration, and sweep families.

When repaired experiments or recovery sessions are cited, the artifact package should preserve three layers of provenance instead of collapsing everything into one synthetic result. First, cite the original experiment directory under Testing/results/experiments/<experiment_id>/. Second, cite the recovery session manifest under recovery_attempts/<session_id>/recovery_session.json, which records which runs and sample-task cases were relaunched. Third, if repaired aggregate figures or summary tables are used, cite the rebuilt repaired experiment directory and its recovery_rebuild_manifest.json. This makes it explicit that a repaired experiment is a derived artifact layered on top of the original sweep, not an in-place

replacement of the original raw results. For older pre-refactor development artifacts that still live in the legacy archive, the same principle applies: cite the archived experiment identifier and note when the thesis figure comes from an archive-only repaired or rebuilt result set rather than from the active results tree.

A.4 Held-Out Failure Accounting

The main results chapter keeps the unrecovered held-out failures brief so that the repaired final comparisons remain central. This appendix section collects the fuller breakdown for readers who want to inspect the one-shot failure pattern directly. The source artifact for these counts is the original unrecovered final-run summary generated alongside `final\_eval\_caveat\_summary.json` and its companion figures.

These appendix figures support two narrow conclusions. First, the original one-shot robustness problem was real and was concentrated on a subset of the harder held-out executables rather than evenly distributed across the whole benchmark. Second, the dominant failure mode was infrastructural: most failed worker assignments overflowed the model context window during Ghidra-heavy work. They should not be read as proof that the underlying analytical direction was uniformly poor on those samples.

A.5 Detailed Final Per-Executable Panels

The main results chapter keeps the heatmap view as the primary executable-level figure because it is the clearest way to show that the repaired final comparison is heterogeneous across samples. The regenerated paged panels below are included for traceability. They preserve the repetition-level score variance for each configuration on each executable without compressing all five samples into one unreadable composite.

Sample	Diff.	Major implemented methods or intended challenge
`config_decoder_test_stripped.exe`	hard	Stripped decoder sample with single-byte XOR configuration recovery, parser logic, and validation branching; stresses recovery of decode and parser behavior without preserved symbol names.
`config_decoder_test_upx_stripped.exe`	hard	Same underlying decoder logic under UPX packing and stripped symbols; adds packer triage and unpack-aware reasoning before control-flow and data-flow recovery.

Table 1.2: Executable set used by the repeated and follow-up decoder-depth studies.

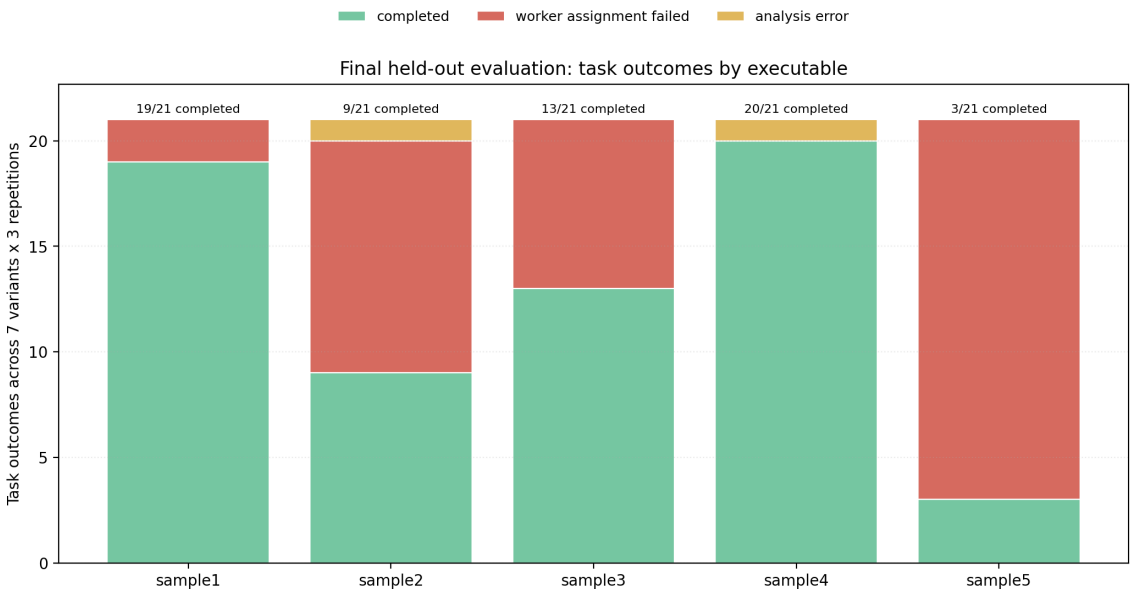

Figure 1.1: Outcome mix by executable in the original unrecovered held-out sweep. The main point of this appendix figure is that the failure burden was not evenly distributed across the five held-out samples.

A.6 Reproduction Commands

This section collects the repository commands used to rebuild corpora, run preflight checks, and launch the maintained experiment presets. The main text cites preset names and artifact locations, while the full command lines live here so the body chapters remain focused on methodology and interpretation.

Broad Coverage Pilot

Rebuild and prepare the experimental corpus:

```
make all-exes
python Testing/scripts/prepare_bundles.py --corpus experimental
```

Preflight and launch the broad pilot:

```
python Testing/scripts/run_launch_preset.py \
  --preset coverage_broad_r1_60usd \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset coverage_broad_r1_60usd \
  --judge-model openai:gpt-4o-mini
```

Focused Decoder-Depth Studies

The repeated decoder-depth suite:

```
python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_r3 \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_r3 \
  --judge-model openai:gpt-4o-mini
```

The narrower follow-up depth suite:

```
python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_followups_r3 \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset sweep_decoder_depth_followups_r3 \
  --judge-model openai:gpt-4o-mini
```

Final Held-Out Evaluation

Rebuild and prepare the final corpus:

```
make deepseek-final
python Testing/scripts/prepare_bundles.py --corpus final_round
python Testing/scripts/run_launch_doctor.py --corpus final_round --bundle-only
```

Preflight and launch the held-out evaluation:

```
python Testing/scripts/run_launch_preset.py \
  --preset thesis_final_eval_r3_100usd \
  --judge-model openai:gpt-4o-mini \
  --preflight-only

python Testing/scripts/run_launch_preset.py \
  --preset thesis_final_eval_r3_100usd \
  --judge-model openai:gpt-4o-mini \
  --live-view \
  --max-concurrent-child-runs 9 \
  --max-concurrent-repetitions 3 \
  --enable-budget-guardrails
```

Preflight-only artifacts are written under `Testing/results/preflight/`. Executed experiments write canonical run catalogs, child runs, summaries, and figures under `Testing/results/experiments/<experiment\_id>/`.

Sample	Evaluator family	Major implemented methods or intended challenge
`sample1.exe`	Emotet simulation	Anti-analysis gating, run-key persistence, encoded configuration recovery, network staging, and process injection; tests whether the workflow can ground persistence and injection claims without overstating full real-world protocol coverage.
`sample2.exe`	QakBot simulation	Mode-driven execution, scheduled-task persistence, resource-backed payload staging, injection, named-pipe coordination, and self-delete support logic; tests careful reporting when resource blobs are intentionally incomplete.
`sample3.exe`	TrickBot simulation	Anti-analysis, integrity checking, registry persistence, encrypted C2 recovery, and staged post-exploitation behavior; tests whether network and banking-style claims stay tied to recovered encrypted configuration and code-level anchors.
`sample4.exe`	WannaCry simulation	Killswitch gating, service persistence, worm-like SMB scanning, ransomware presentation, and resource extraction; tests whether the workflow explains both ransomware and propagation behavior while noting missing-resource limits.
`sample5.exe`	PlugX simulation	Registry persistence, configuration decryption, runtime profile decoding, C2 heartbeat framing, and plugin-style RAT behavior; tests long-horizon configuration recovery and profile interpretation under a more modular RAT-style structure.

Table 1.3: Blind held-out executable set used in the maintained final evaluation.

Stage	Prompt emphasis	Expected structured output
Preflight	minimal reconnaissance only; resolve the sample path, basic metadata, and a few worker pivots without deep analysis	short handoff note for planning
Deterministic presweeps	bounded reusable collection such as strings, FLOSS, capa, packer checks, UPX attempts, and compact Ghidra metadata	compact reusable sweep bundle
Planner	planning only, no fresh analysis; convert the request and shared context into executable work items	JSON work-item block between `PLANNER_WORK_ITEMS_START` and `PLANNER_WORK_ITEMS_END`
Workers	evidence-backed specialist analysis, role-specific bounded work, optional Ghidra or YARA proposal output	structured worker evidence bundle plus optional machine-readable proposal blocks
Validators	review only, no broad new discovery; decide whether the answer is adequate and what should be narrowed or retried	JSON validation decision block between `VALIDATION_DECISION_START` and `VALIDATION_DECISION_END`
Reporter	synthesis only; no fresh tool use; final answer must stay tied to validated findings and meaningful control-flow pivots	concise final analyst report and any finalized proposal blocks

Table 1.5: The main prompt and contract surfaces used by the maintained multi-agent workflow.

Original unrecovered held-out outcome	Count
Sample-task evaluations attempted	105
`completed`	64
`worker_assignment_failed`	39
`analysis_error`	2

Table 1.6: Task-level outcome totals in the original unrecovered held-out sweep.

Worker-assignment failure class	Count
Failed worker assignments observed	58
`context_length_exceeded`	49
`invalid_request_payload`	6
`tool_retry_exhausted`	2
`usage_limit_exceeded`	1

Table 1.7: Assignment-level failure classes in the original unrecovered held-out sweep.

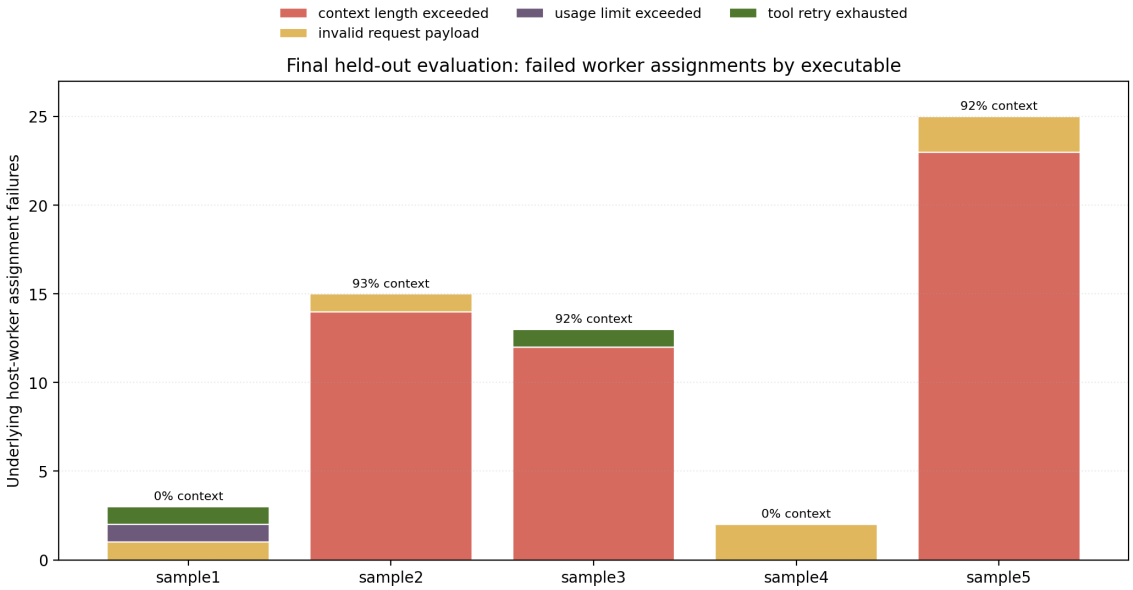

Figure 1.2: Underlying worker-assignment failure causes by executable in the original unrecovered held-out sweep. Most failures came from host-worker context-window overflow on a subset of samples, with a smaller tail of malformed-request, tool-retry, and request-budget failures.

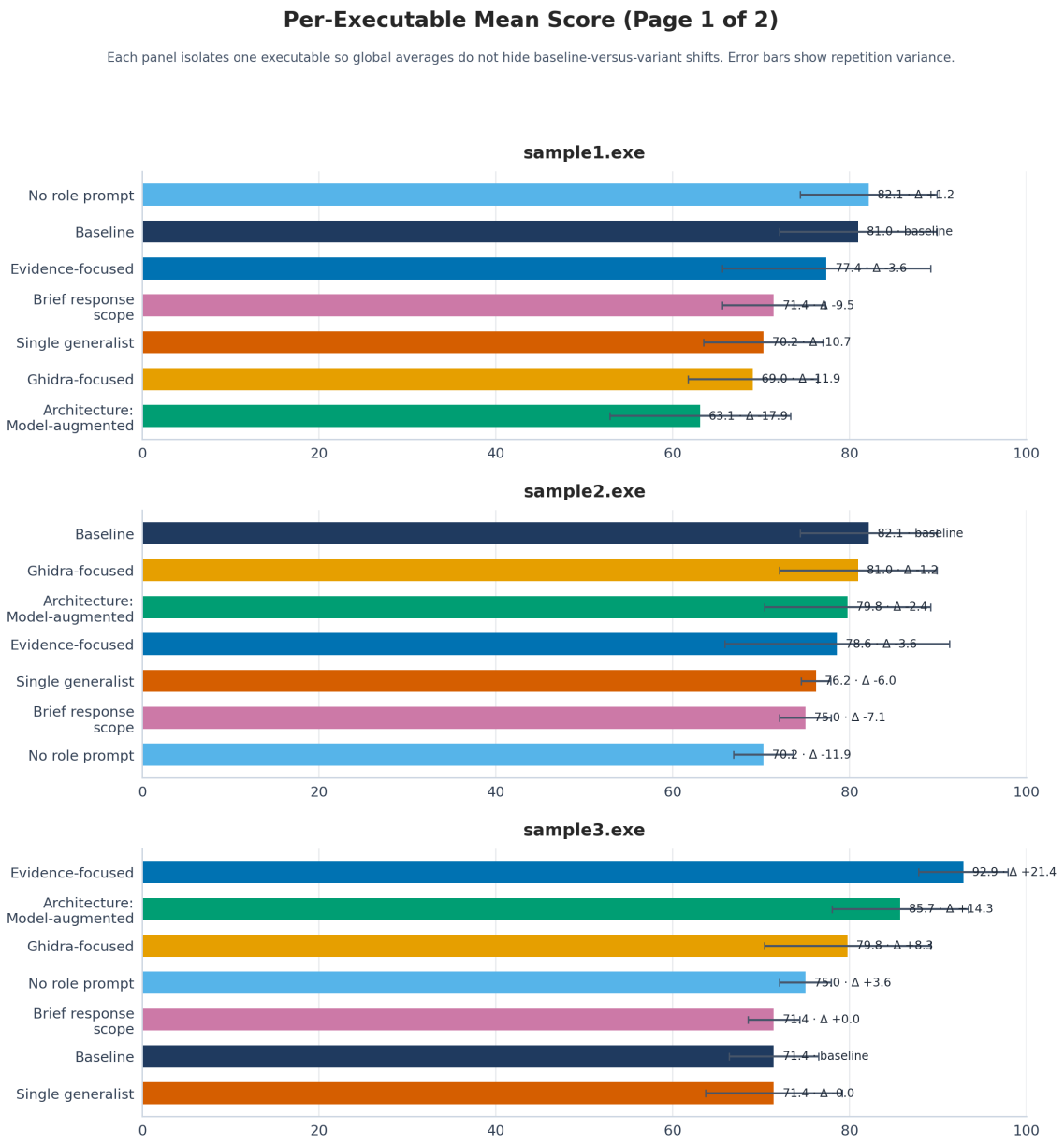

Figure 1.3: Paged per-executable score panels for the repaired final held-out experiment, page 1. This page covers `sample1.exe` through `sample3.exe` and shows the mean score plus repetition-level variance for each maintained configuration.

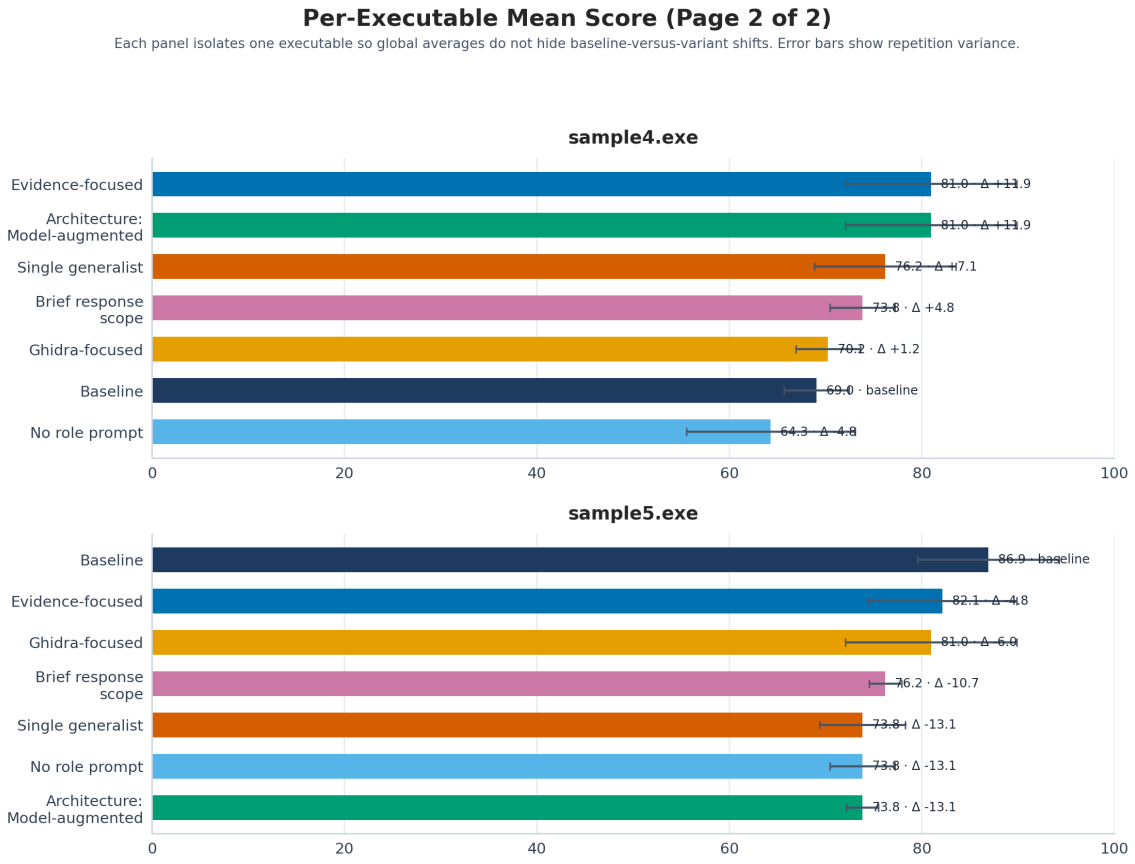

Figure 1.4: Paged per-executable score panels for the repaired final held-out experiment, page 2. This page covers `sample4.exe` and `sample5.exe` and completes the executable-level detail behind Figure Figure 7.6.